



Распределенное обучение: Model Parallelism

Лекция 9
2026, Дмитрий Нестеров

План лекции

01

Pipeline Parallelism

Naive & Micro-Batch



02

Tensor Parallelism

На примере MLP & Self-Attention



03

Sequence Parallelism

Как считать Self-Attention?



04

Гибридный параллелизм

2D, 3D, 5D



05

Offloading

ZeRO-Offload, ZeRO-Infinity



План лекции

01

Pipeline Parallelism

Naive & Micro-Batch

02

Tensor Parallelism

На примере MLP & Self-Attention

03

Sequence Parallelism

Как считать Self-Attention?

04

Гибридный параллелизм

2D, 3D, 5D

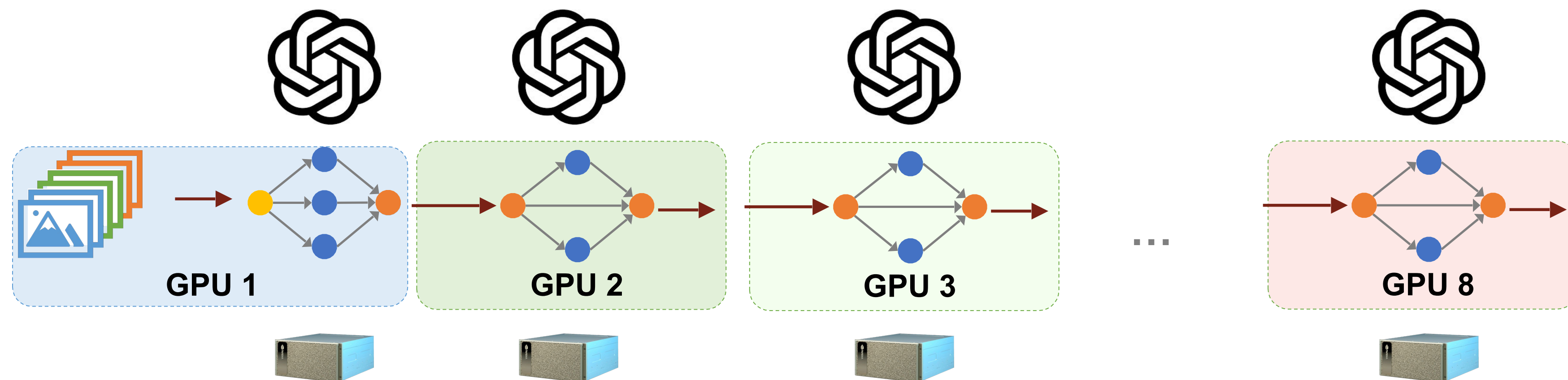
05

Offloading

ZeRO-Offload, ZeRO-Infinity

Pipeline Parallelism

Вместо разделения данных, PP разделяет модель

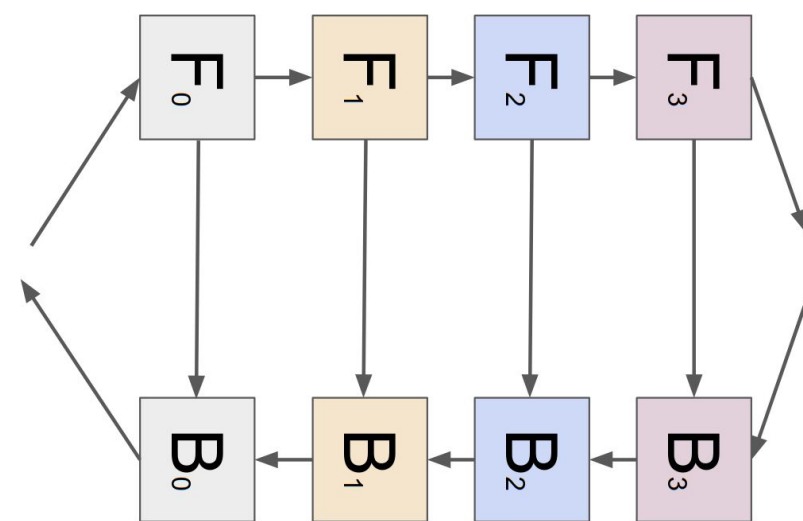


$$350\text{GB} / 8 \text{ GPU} = 43.75\text{G} < 80\text{G}$$

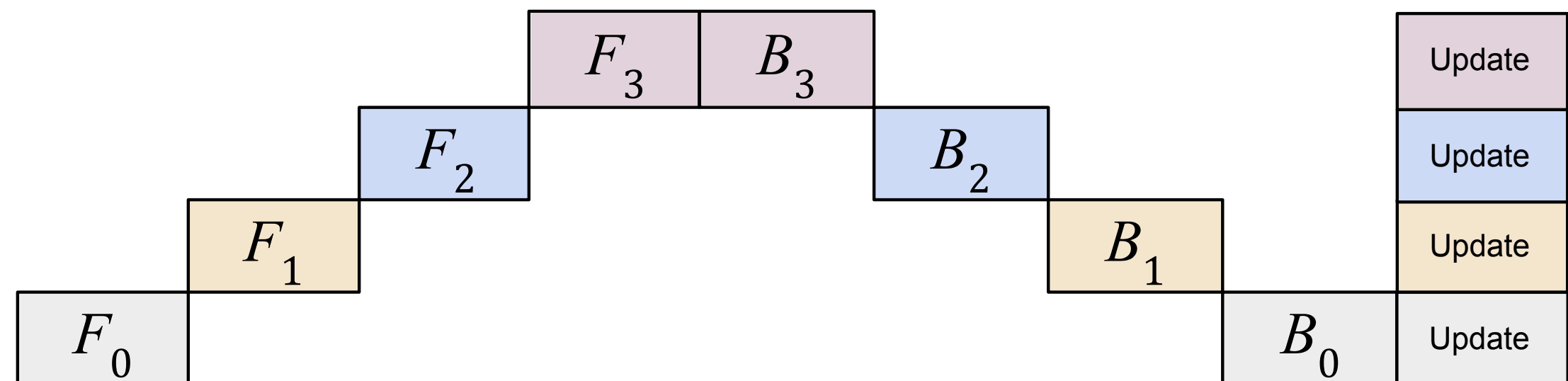
с PP можем обучать большие ML модели

Pipeline Parallelism

Наивная реализация



(a). Training data flow



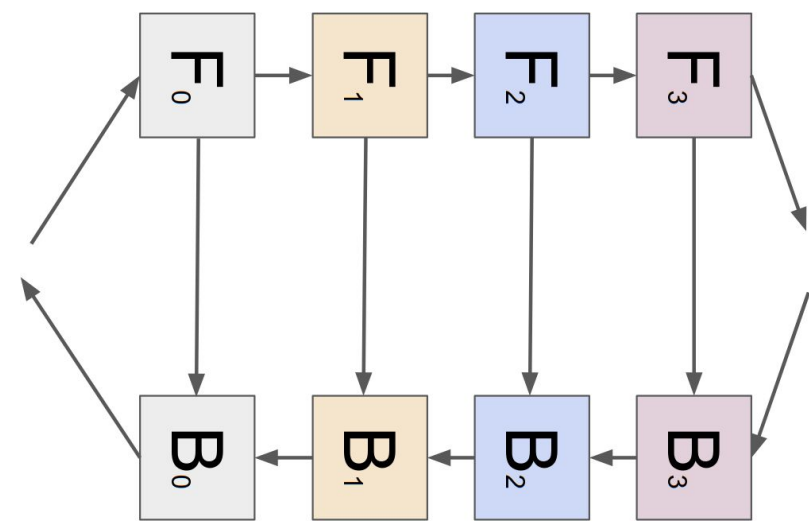
(b). Training timeline

F: Forward B: Backward
Обучаем 4-слойную сеть с PP

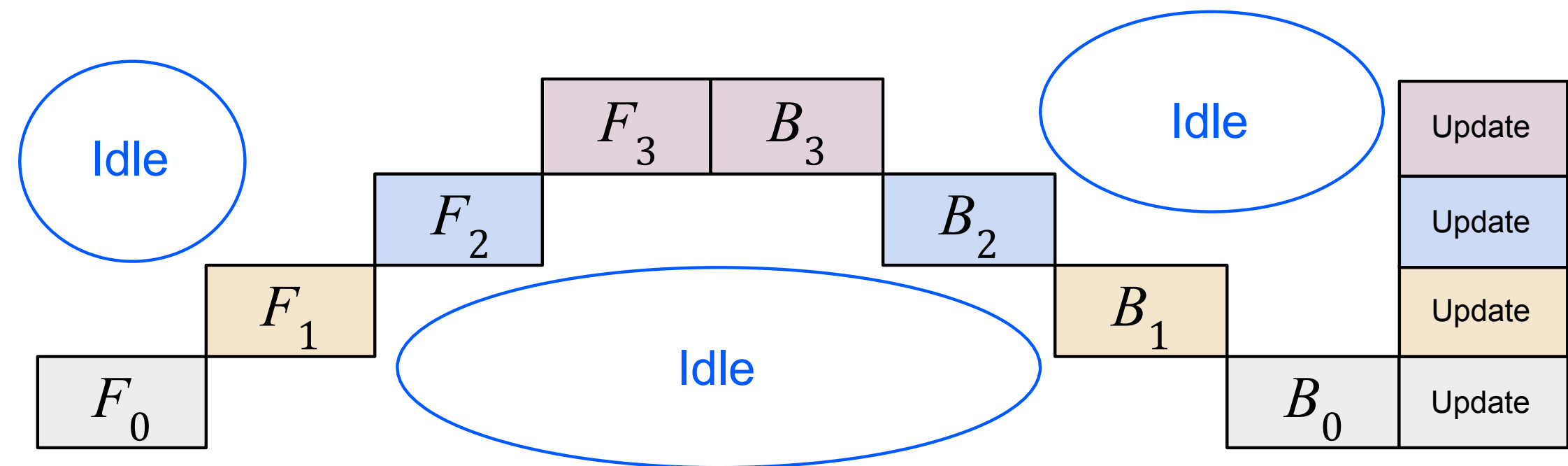
Как работает PP: модель разделяется на части, каждая часть присваивается отдельной GPU

Pipeline Parallelism

Наивная реализация



(a). Training data flow



(b). Training timeline

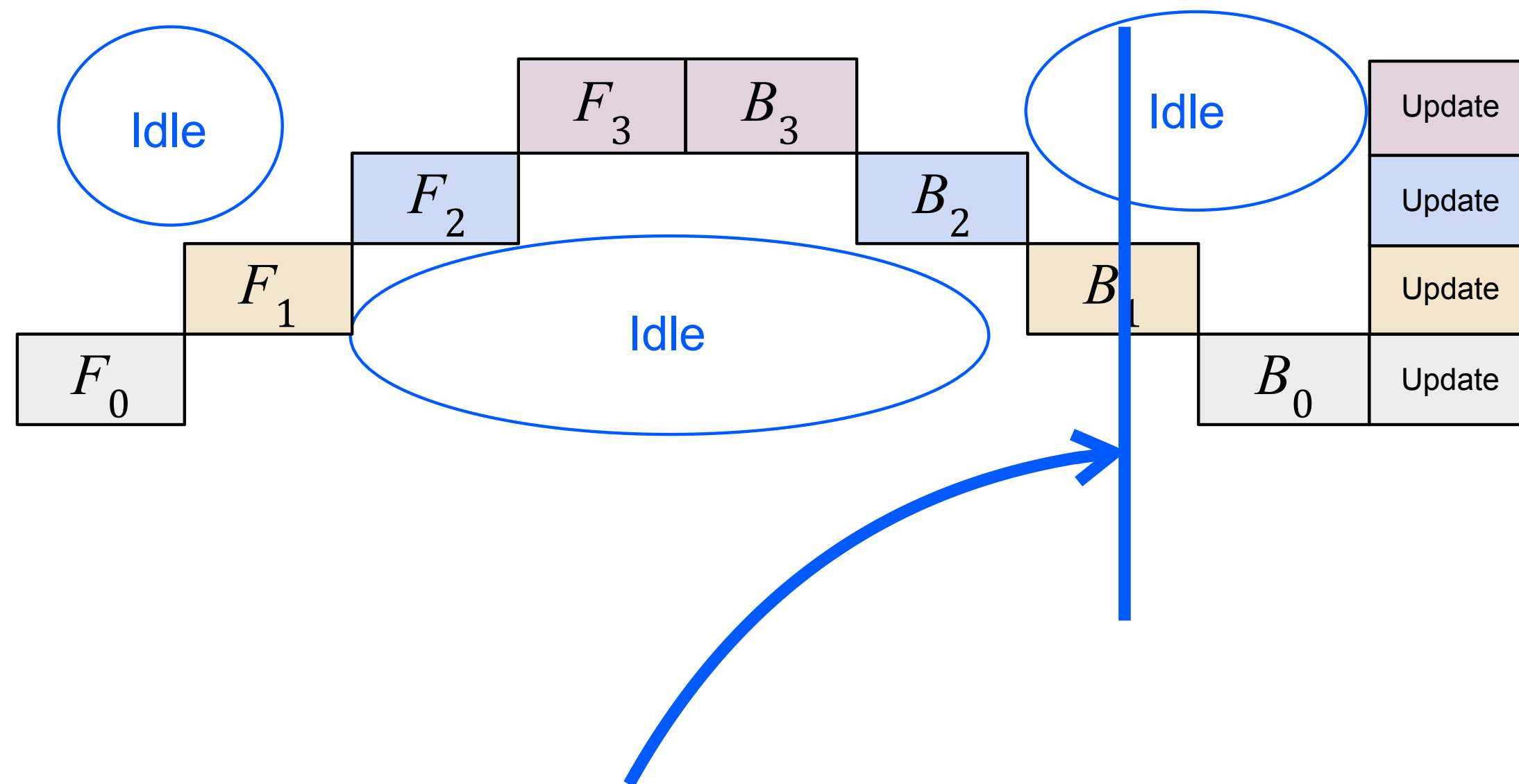
F: Forward B: Backward
Обучаем 4-слойную сеть с PP

Как работает PP: модель разделяется на части, каждая часть присваивается отдельной GPU

Но GPU значительным образом **простаивают!**

Pipeline Parallelism

Наивная реализация страдает от низкой утилизации

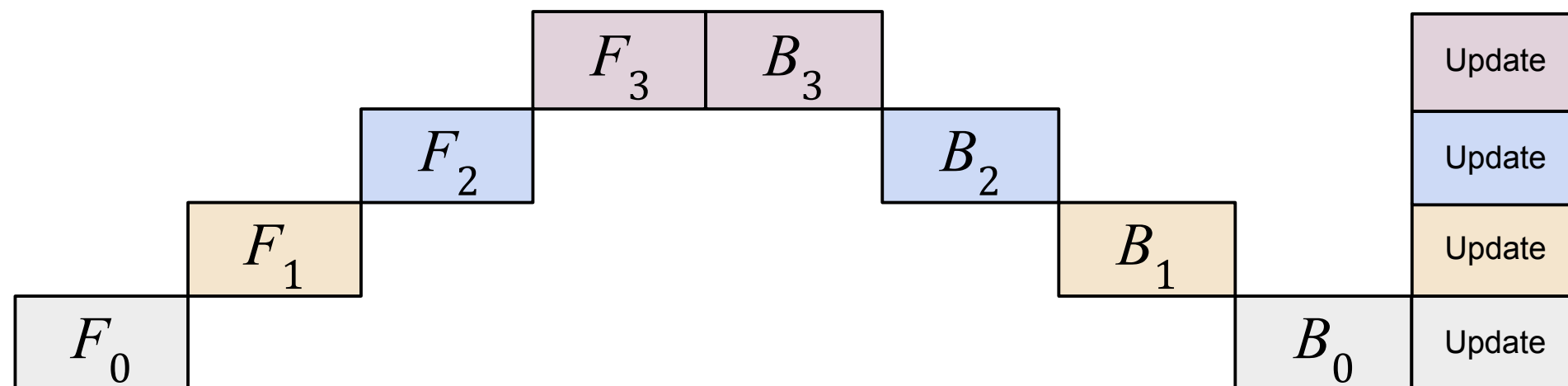


В каждый момент времени вычисления выполняет только один девайс, остальные ждут его

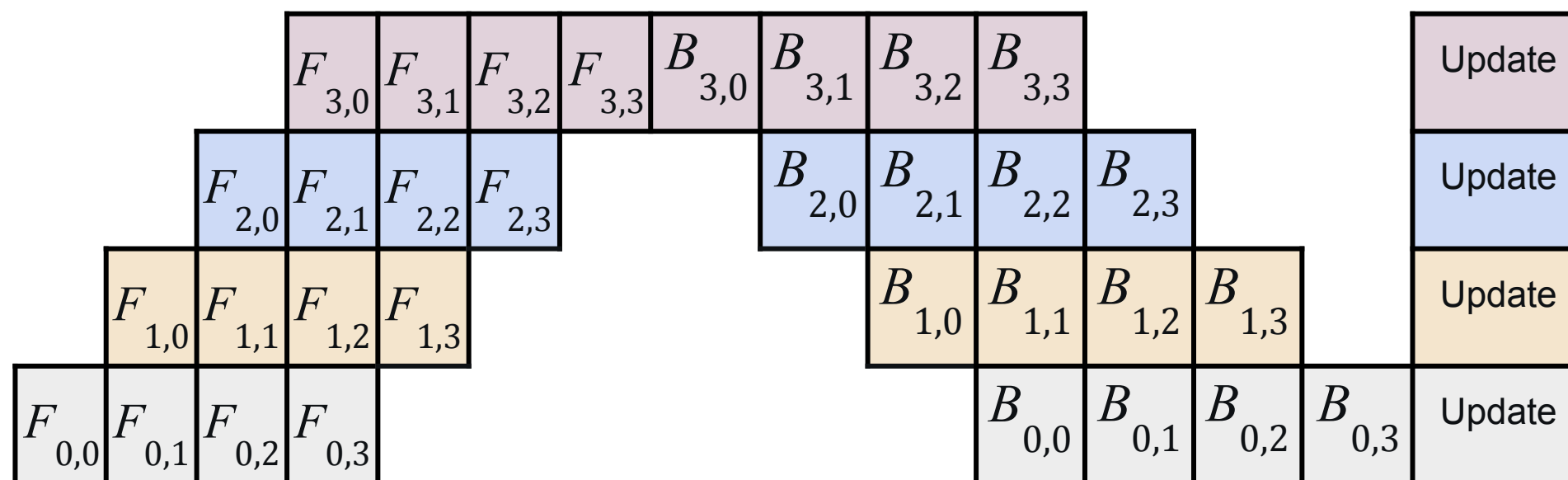
Теоретическая утилизация (в этой 4-layer NN): 25% (**низкая!**)

Pipeline Parallelism

GPipe: Micro-Batch Pipeline Parallelism



(a) Наивный Pipeline Parallelism



(b) Улучшенный Pipeline Parallelism

- Разделяем 1 батч на микро-батчи

- $[16, 10, 512] \rightarrow$

$[4, 10, 512]$

$[4, 10, 512]$

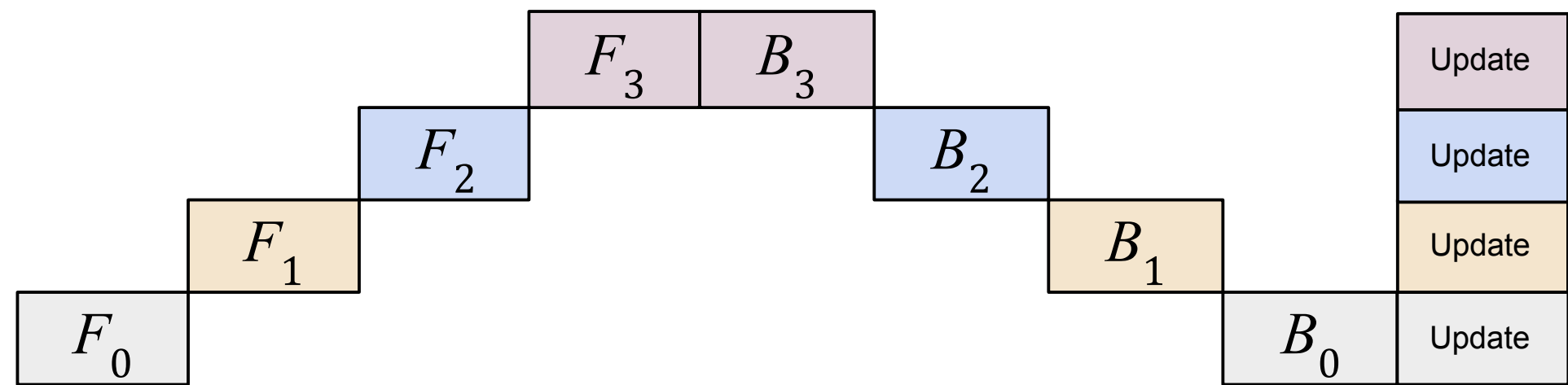
$[4, 10, 512]$

$[4, 10, 512]$

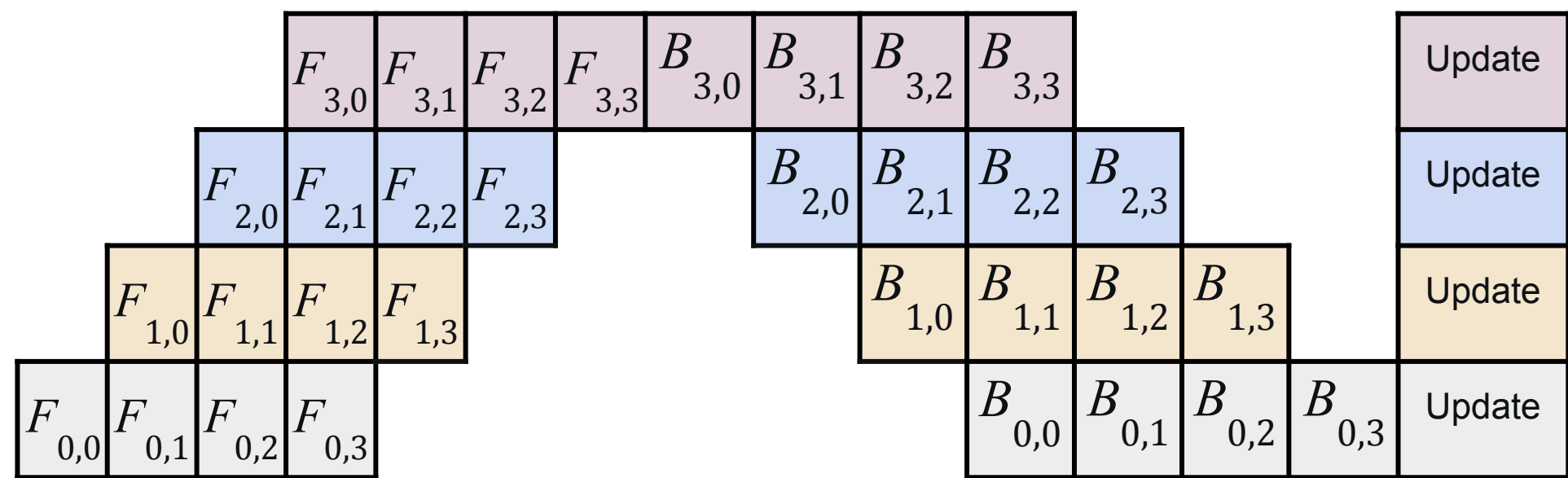
Зачем: параметры модели не меняются при вычислениях в рамках батча $\rightarrow$ поэтому мы можем распараллелить computation и communication

Pipeline Parallelism

Микро-батчинг улучшает утилизацию



(a) Наивный Pipeline Parallelism



(b) Улучшенный Pipeline Parallelism

Утилизация: (25%)

Чем больше микро-батчей, тем выше утилизация

Утилизация: **57% (2.5x)**

План лекции

01

Pipeline Parallelism

Naive & Micro-Batch



02

Tensor Parallelism

На примере MLP & Self-Attention



03

Sequence Parallelism

Как считать Self-Attention?



04

Гибридный параллелизм

2D, 3D, 5D



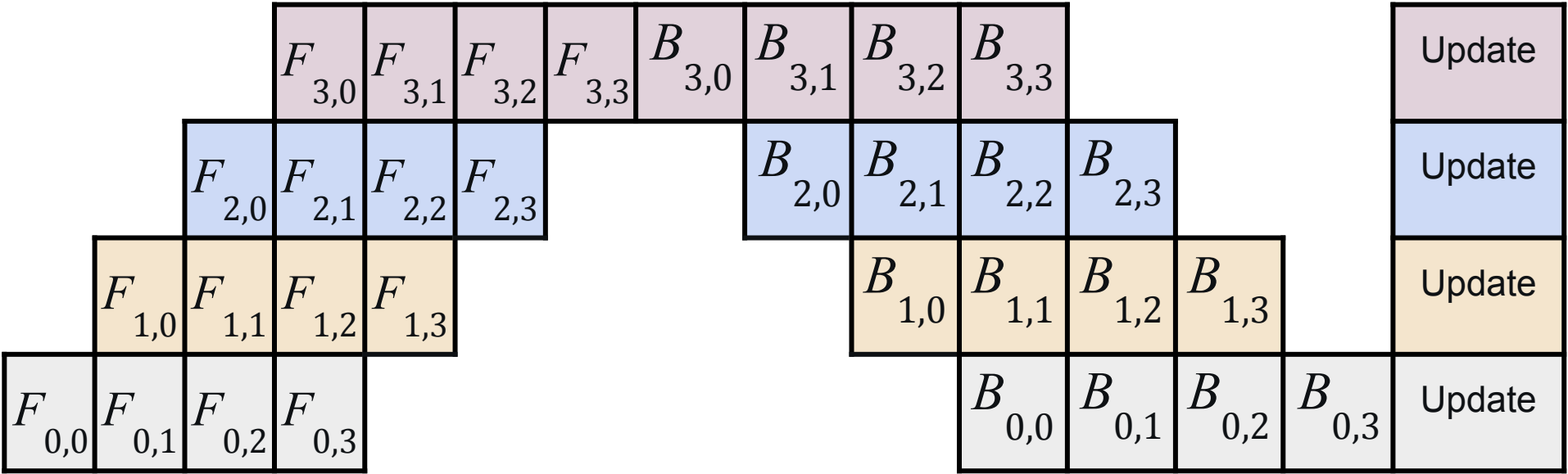
05

Offloading

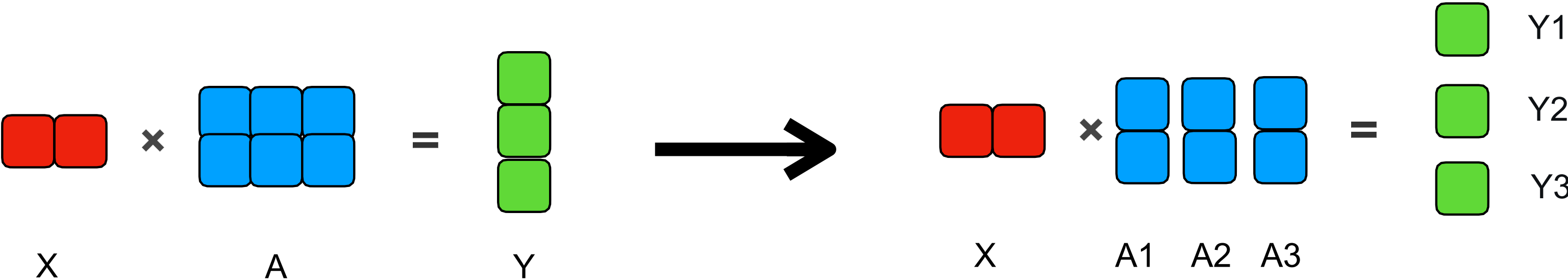
ZeRO-Offload, ZeRO-Infinity



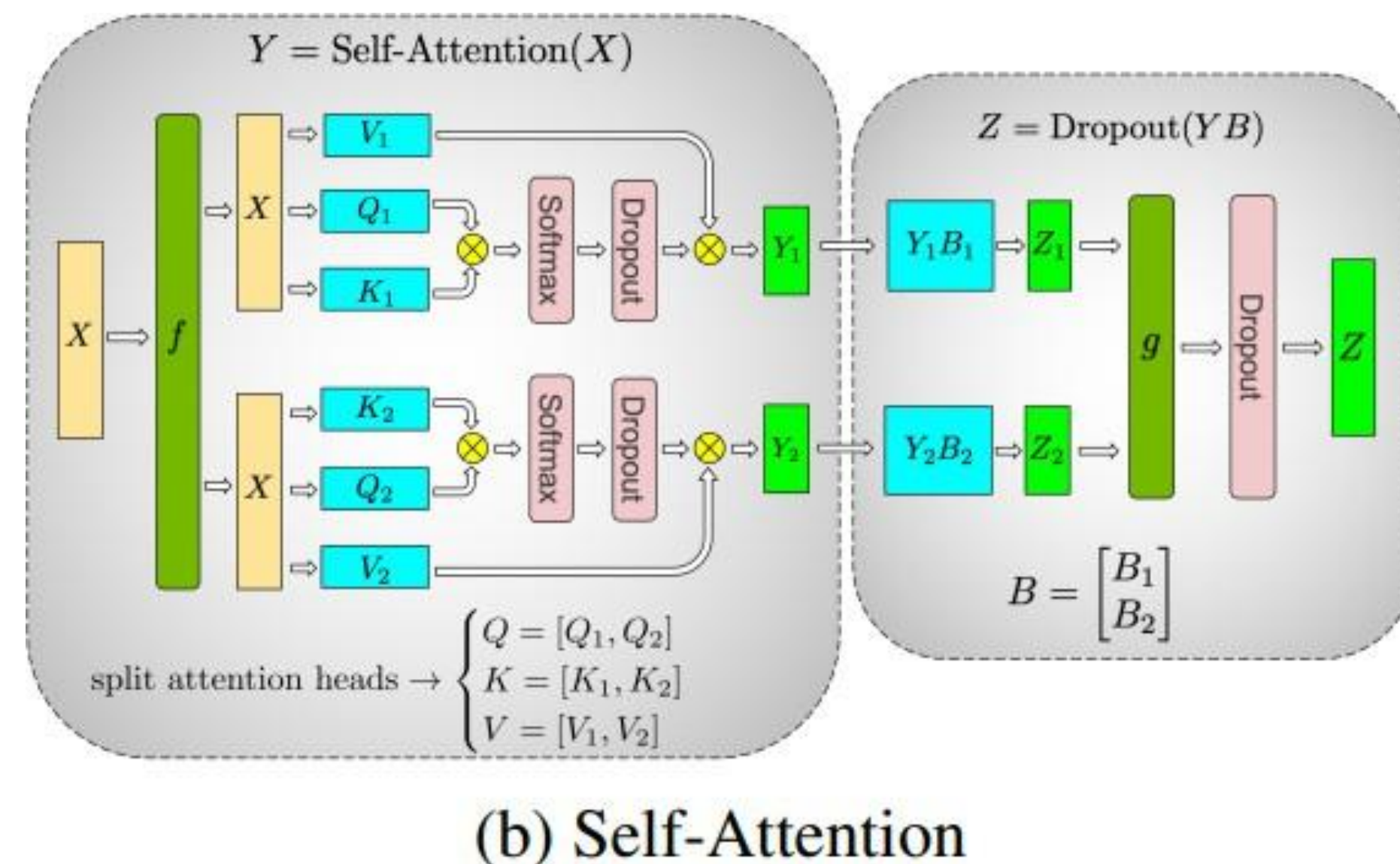
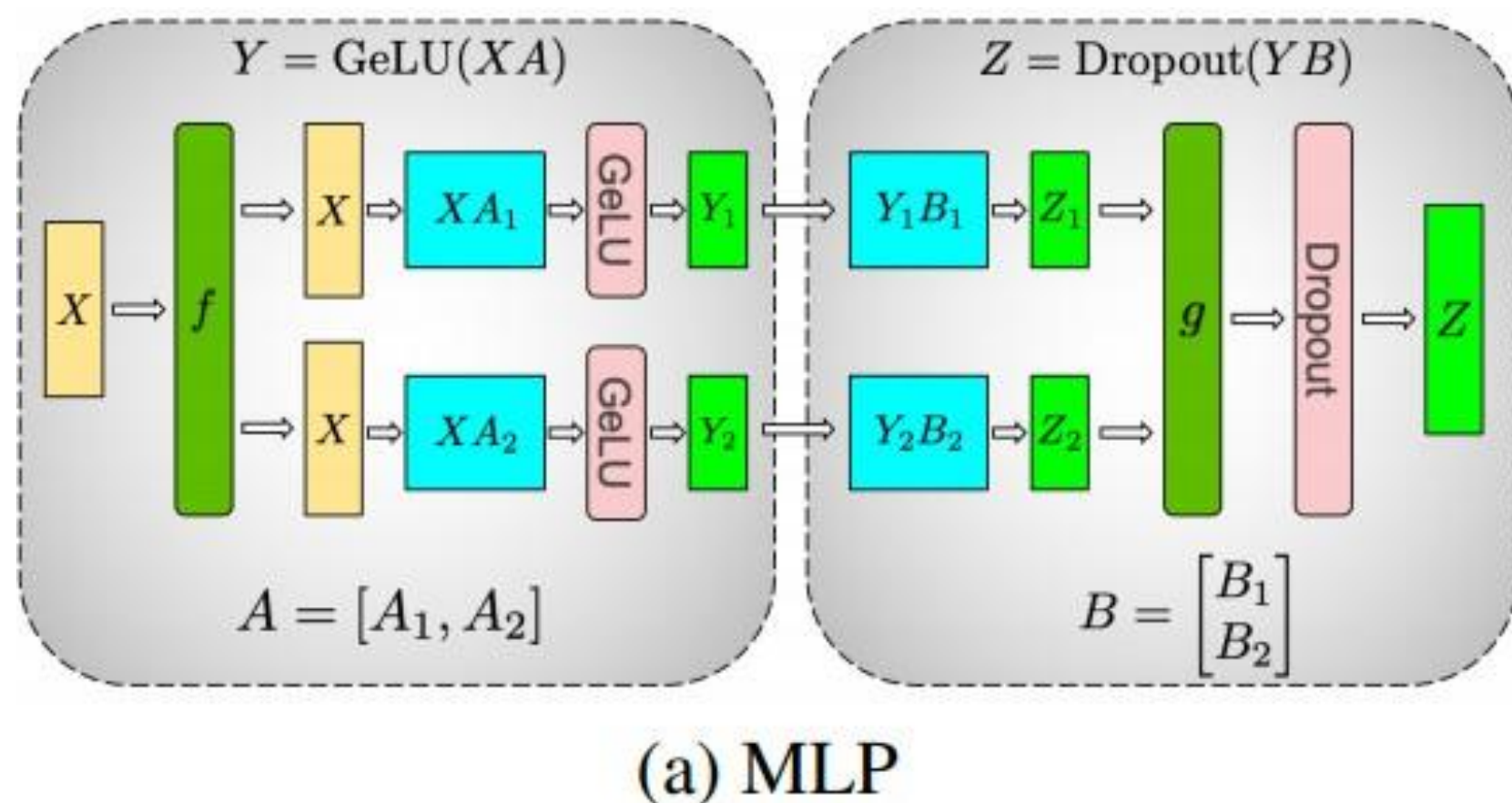
Tensor Parallelism



- GPU все еще простаивают, можно ли это время еще уменьшить?
(сохраняя низкий memory usage)
- Сделаем разбиение модели на части еще **более мелким!**



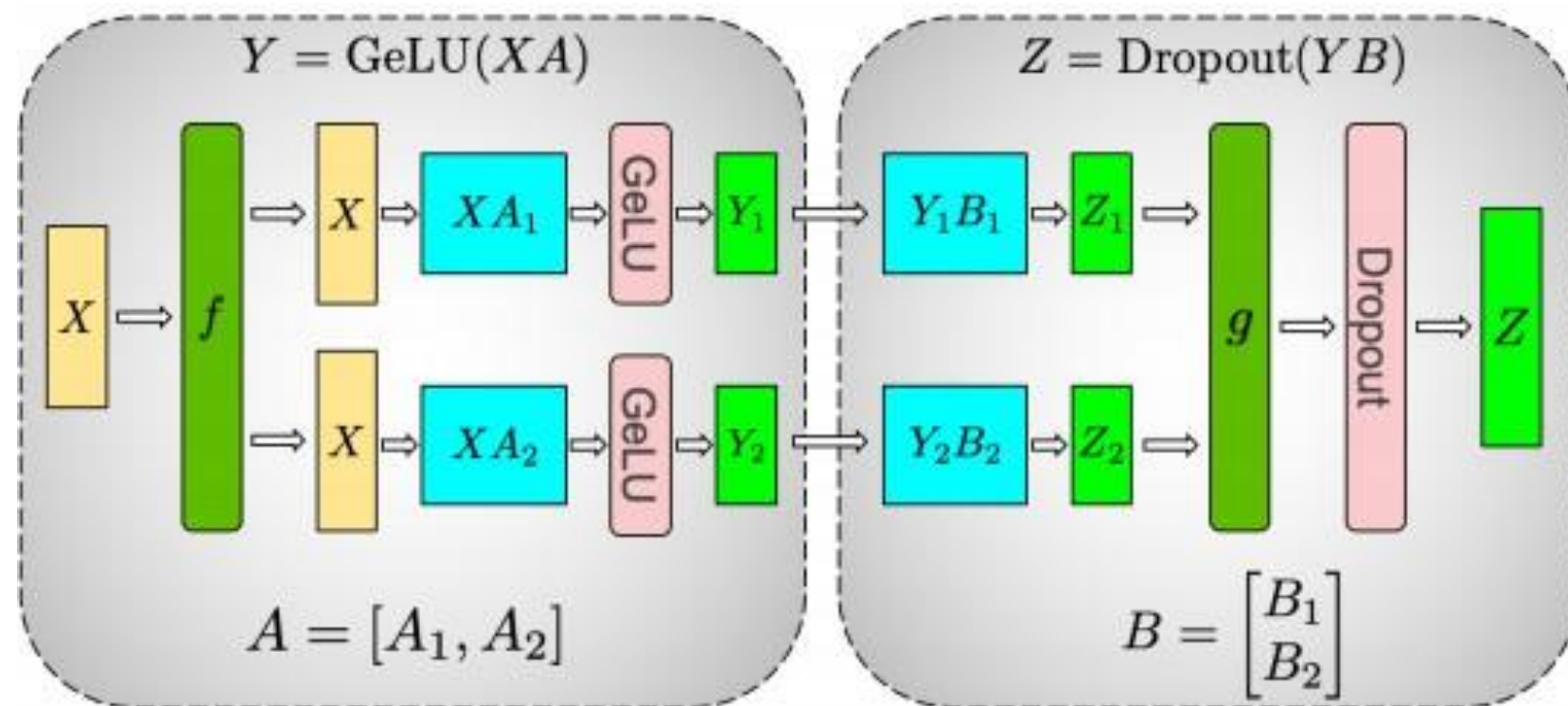
Tensor Parallelism



- Tensor Parallelism: разделяем тензор весов на N частей и параллелизуем вычисления
- f и g — Broadcast / All-Reduce коммуникации
- Параметры и активации распределяются между разными ГПУ

Tensor Parallelism

Разделение в первом FFN слое

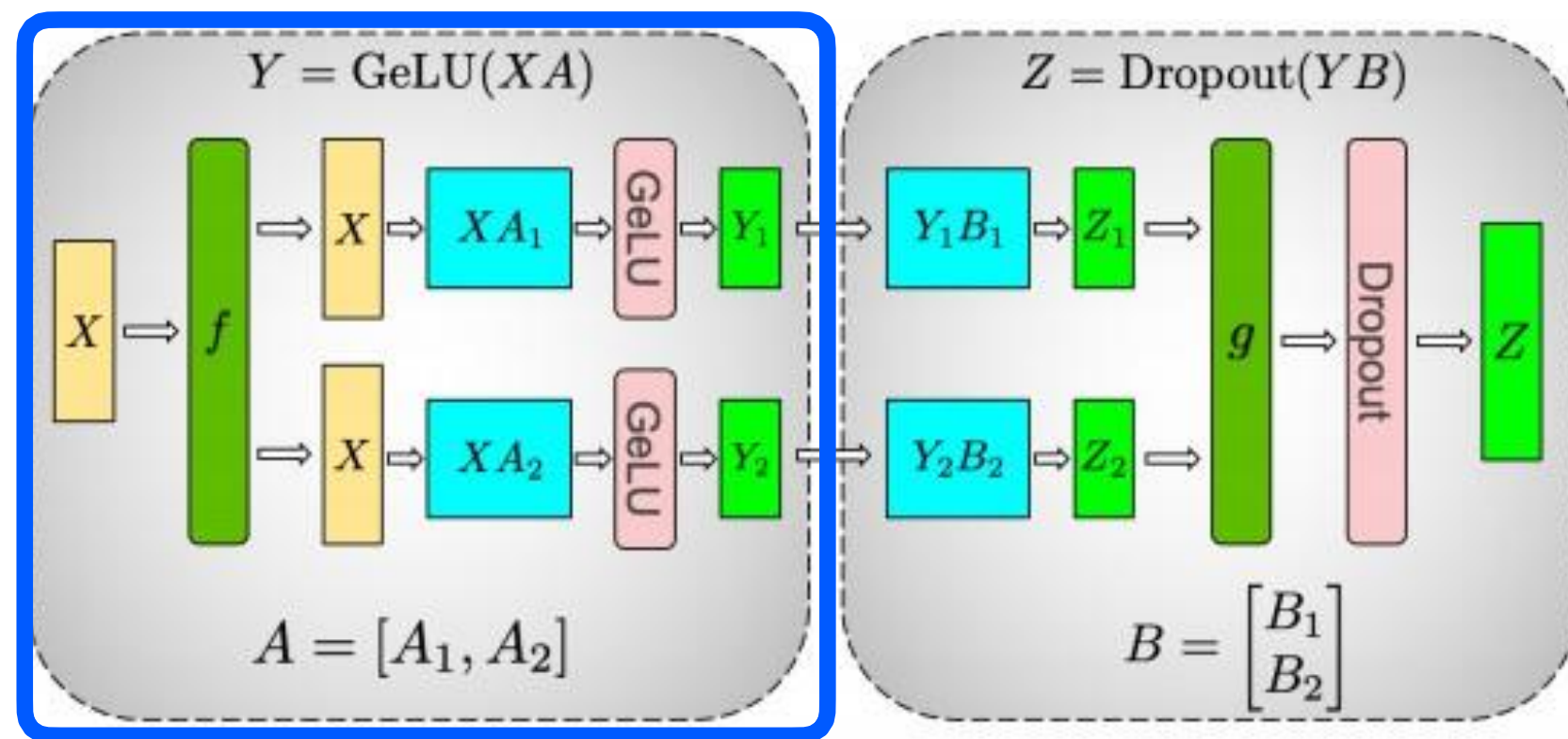


(a) MLP

- В качестве примера рассмотрим $[1, 2] @ [2, 6] @ [6, 2]$

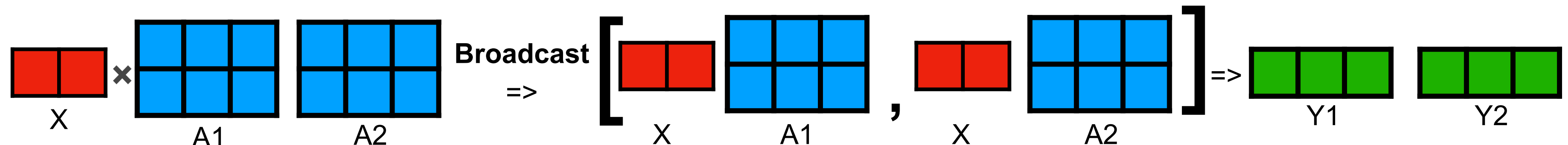
Tensor Parallelism

Разделение в первом FFN слое



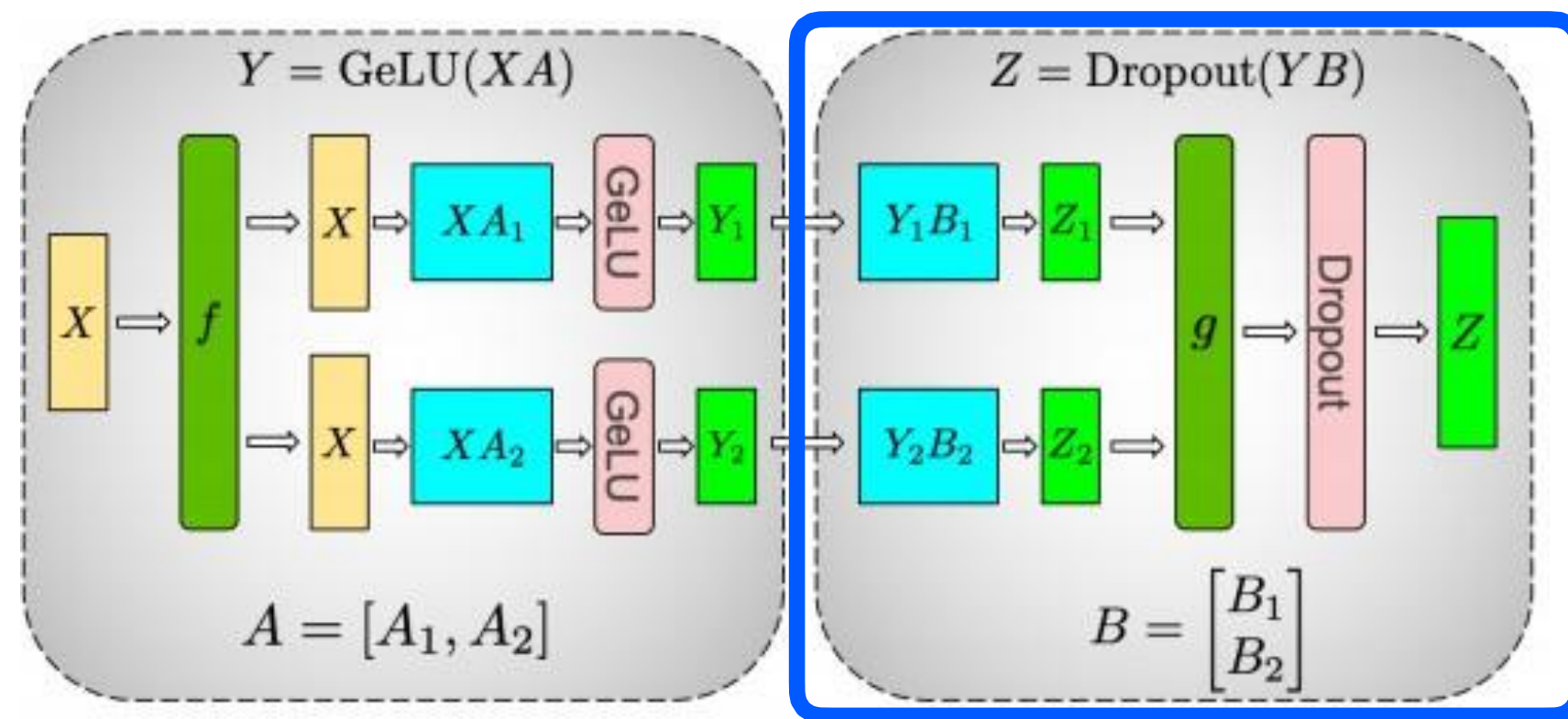
(a) MLP

- В качестве примера рассмотрим $[1, 2] @ [2, 6] @ [6, 2]$
- Первый линейный слой разделен по столбцам
- Используем **Broadcast** для дублирования входа X на все ГПУ



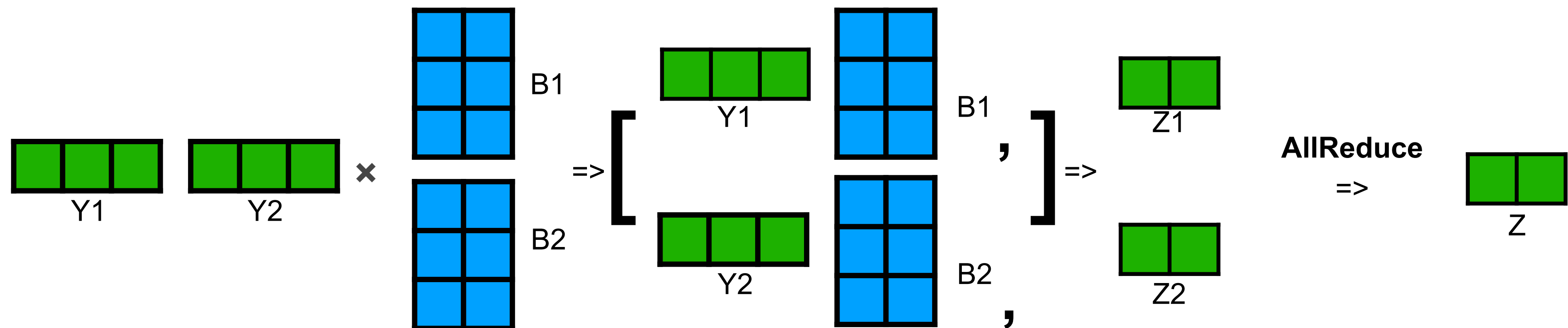
Tensor Parallelism

Разделение во втором FFN слое



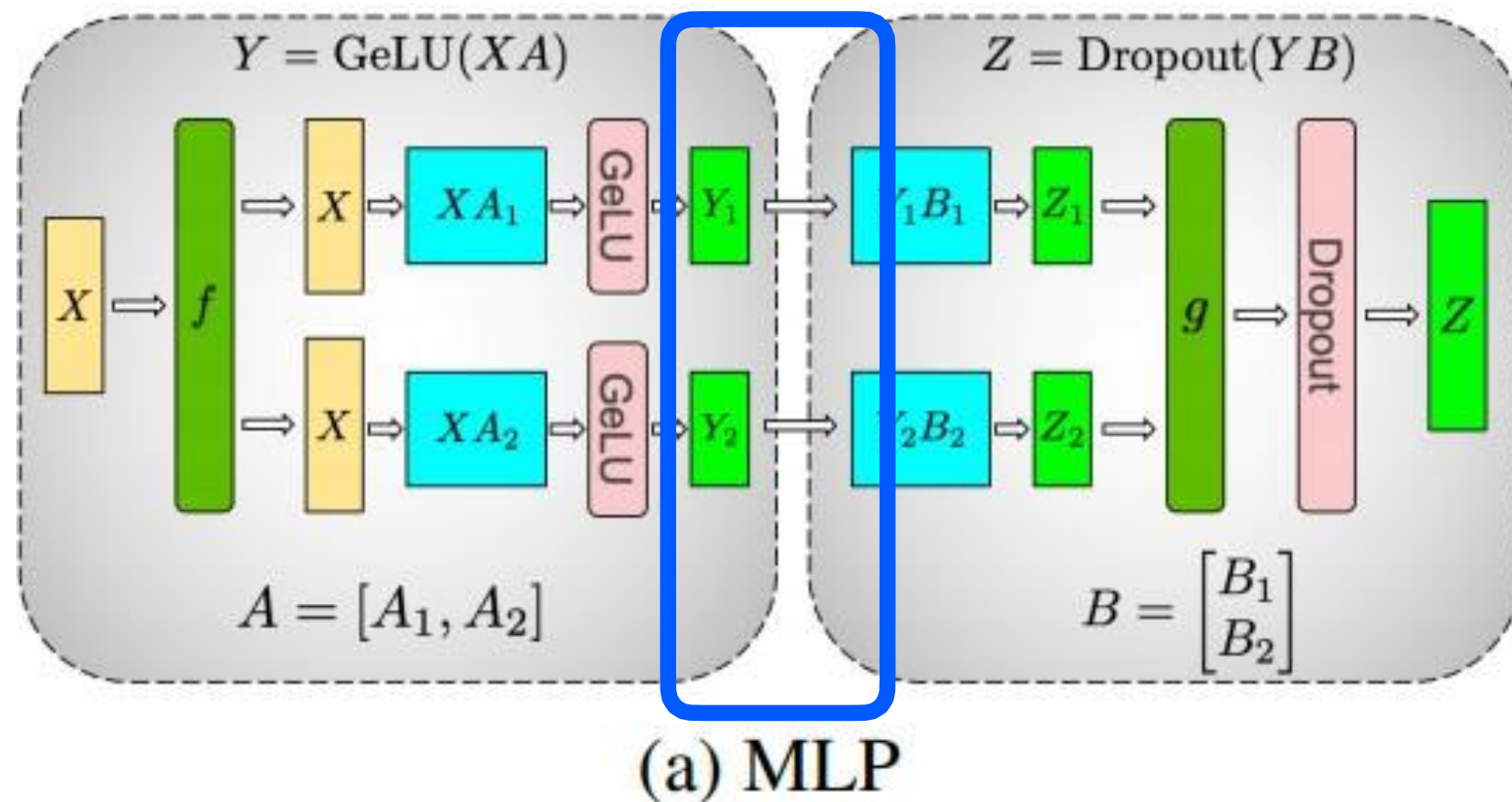
(a) MLP

- В качестве примера рассмотрим $[1, 2] @ [2, 6] @ [6, 2]$
- Второй линейный слой разделен по строкам
- Используем **AllReduce** для агрегации выхода Z



Tensor Parallelism

Разделение в FFN слоях

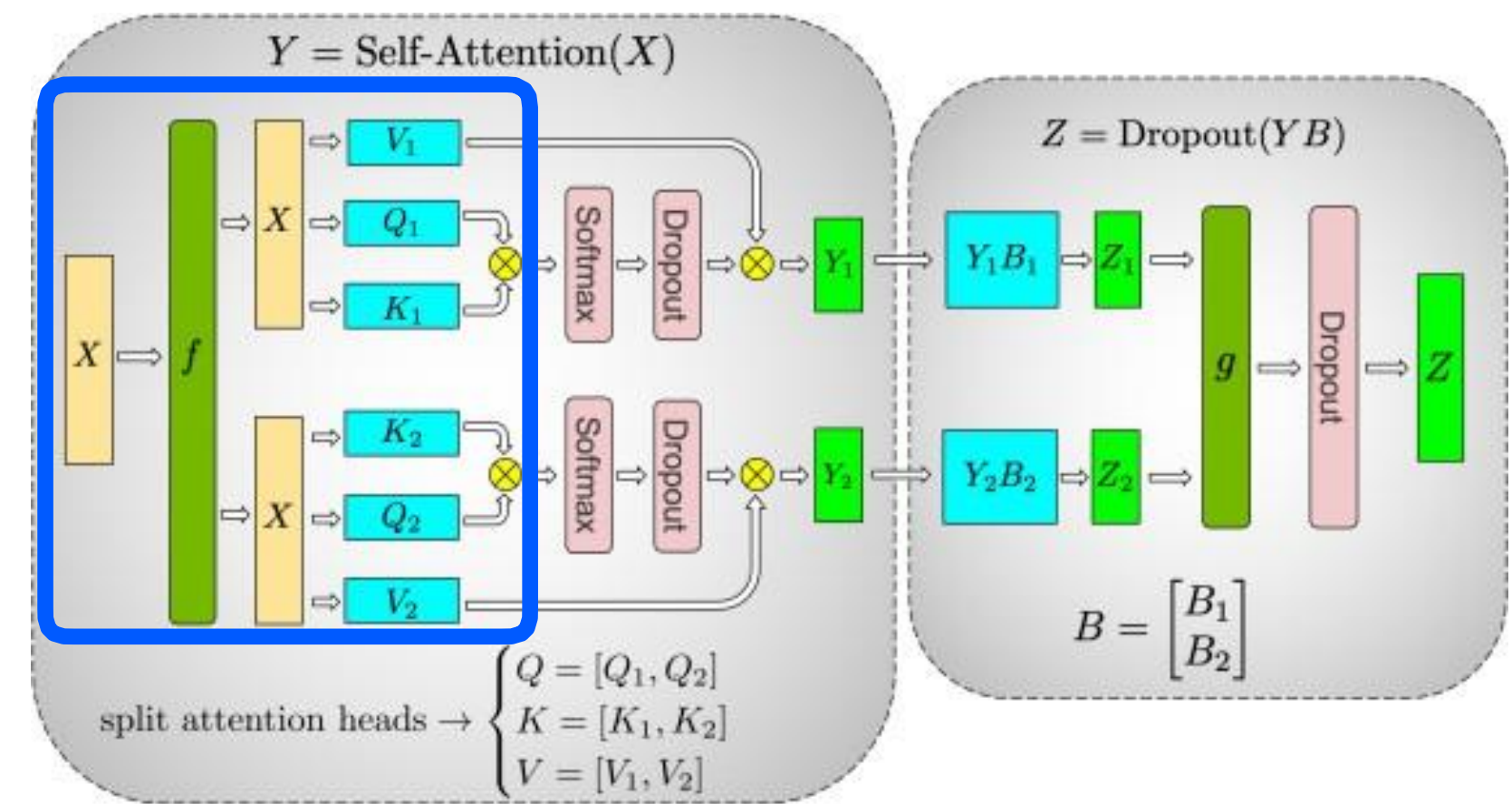


- Разделяем **первое** матричное умножение по **столбцам**, а **второе** по **строкам**
- Поэтому можем избавиться от коммуникаций между слоями
- Утилизируем ГПУ по максимуму, пока сеть не становится узким местом

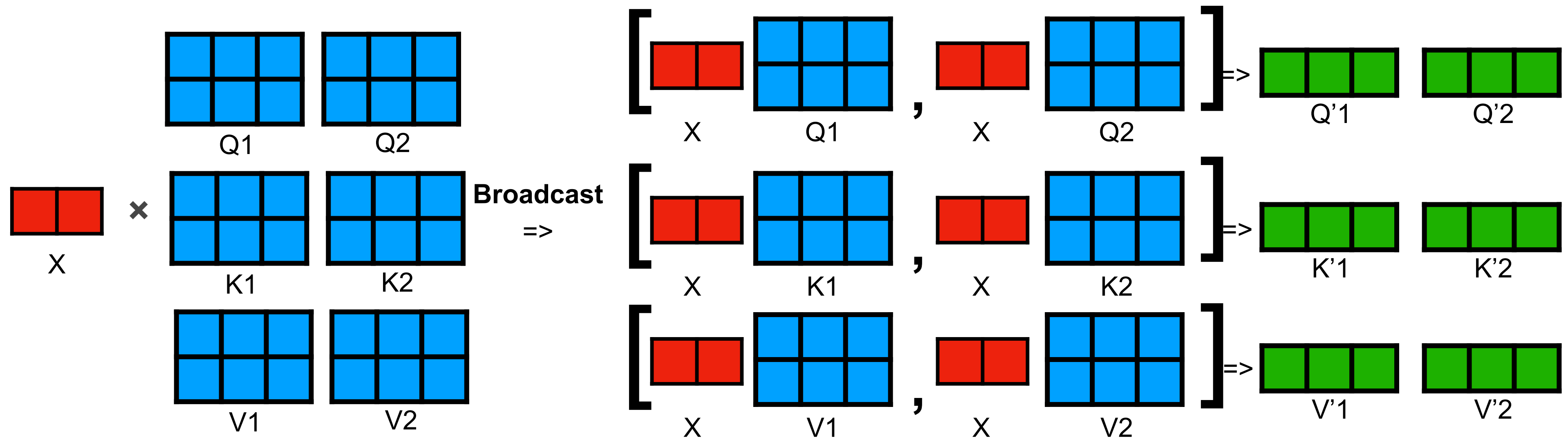
Tensor Parallelism

Разделение Attention: QKV Projection

- Разделяем матрицы QKV по **колонкам**
- Вход **X** дублируем через **Broadcast**



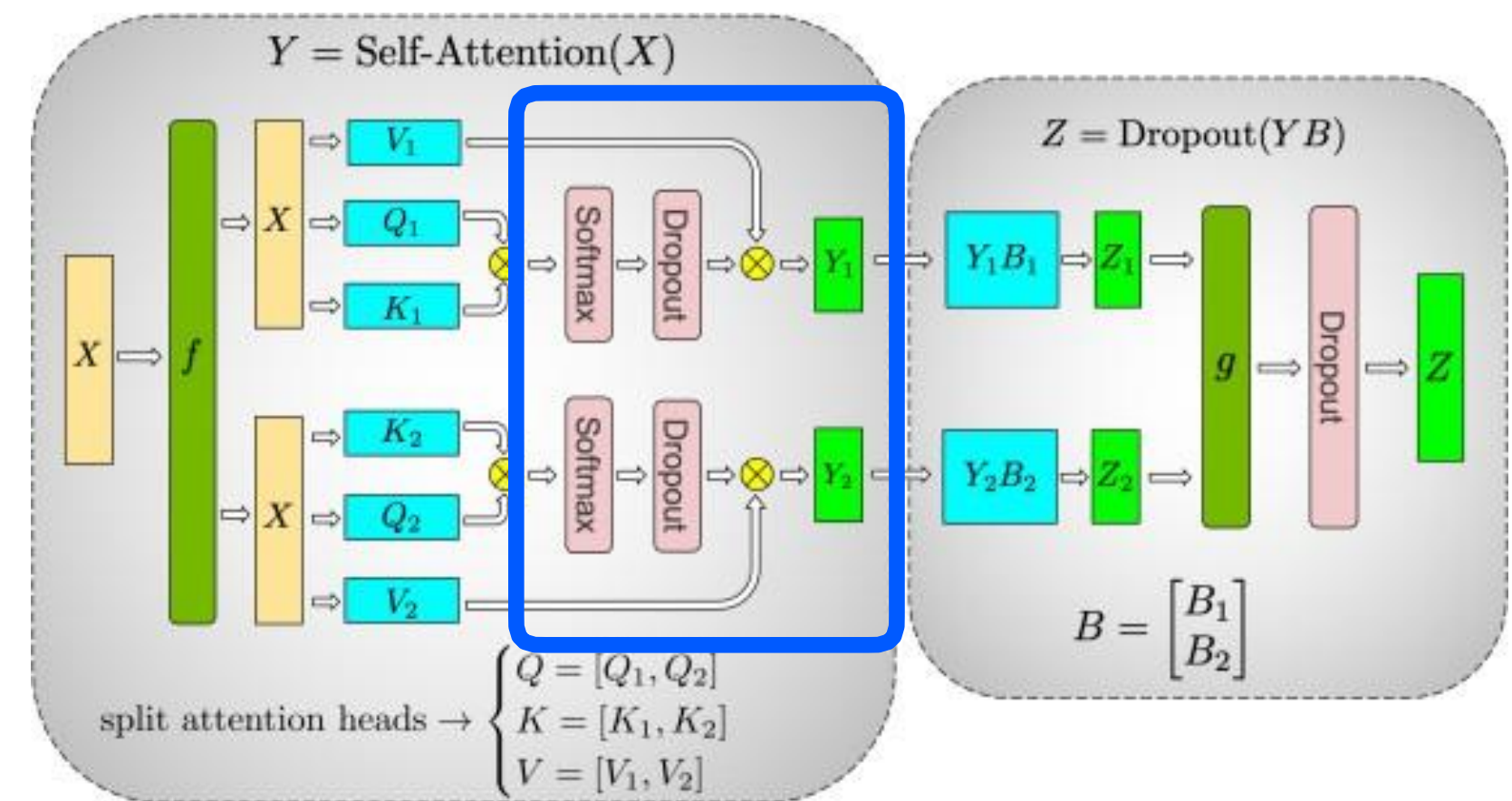
(b) Self-Attention



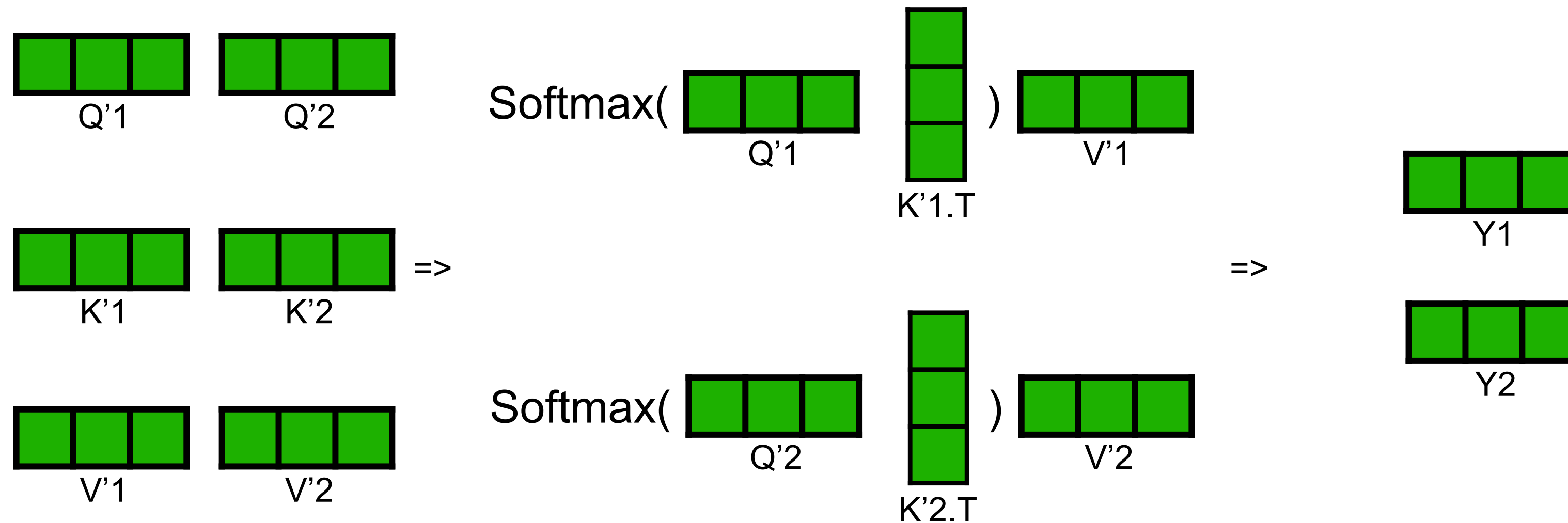
Tensor Parallelism

Разделение Attention: Attention

- $\text{Soft}(Q@K.T)@V$ вычисляется локально без доп. коммуникаций



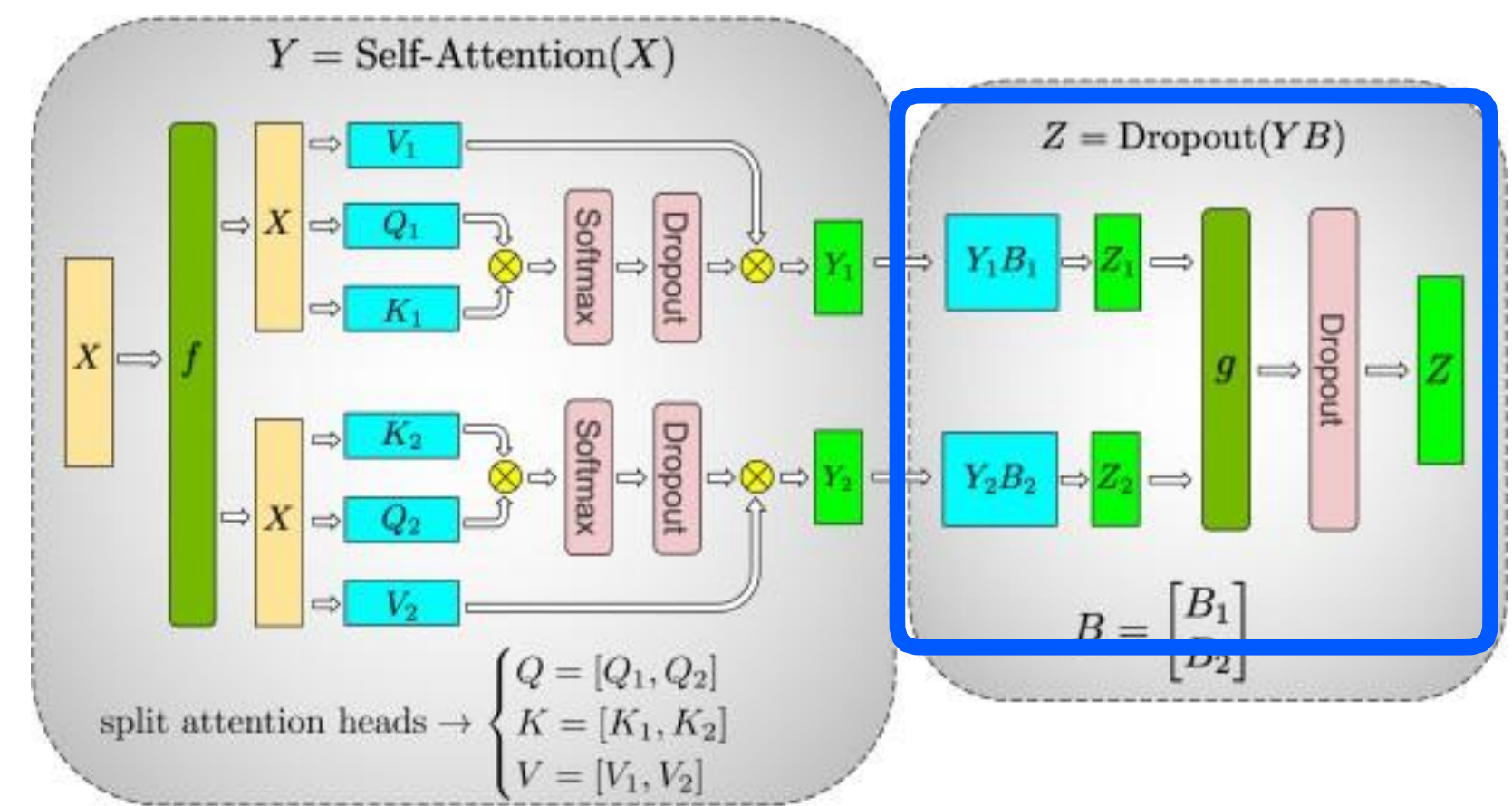
(b) Self-Attention



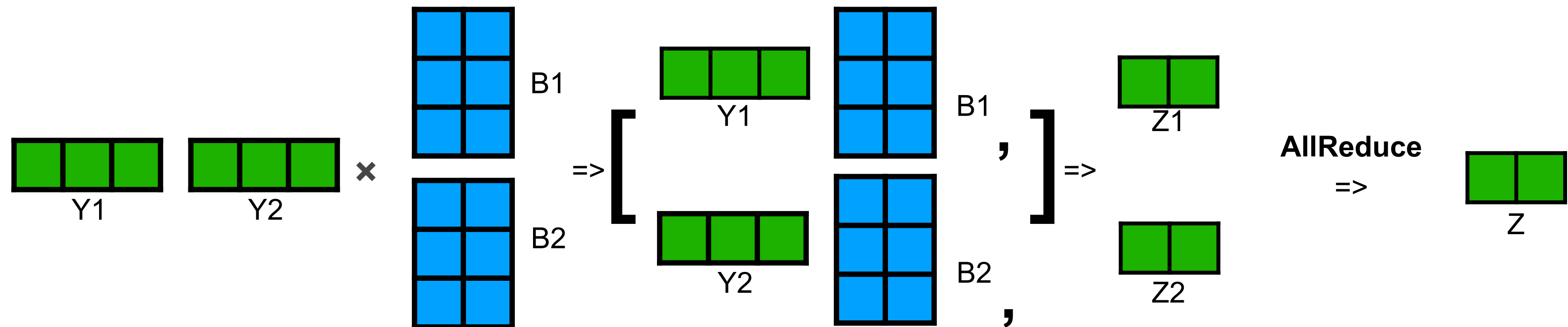
Tensor Parallelism

Разделение Attention: Out Projection

- Разделяем слой по **строкам**
- => Можем произвести вычисления локально
- Собираем выход через **All-Reduce**



(b) Self-Attention



План лекции

01

Pipeline Parallelism

Naive & Micro-Batch



02

Tensor Parallelism

На примере MLP & Self-Attention



03

Sequence Parallelism

Как считать Self-Attention?



04

Гибридный параллелизм

2D, 3D, 5D



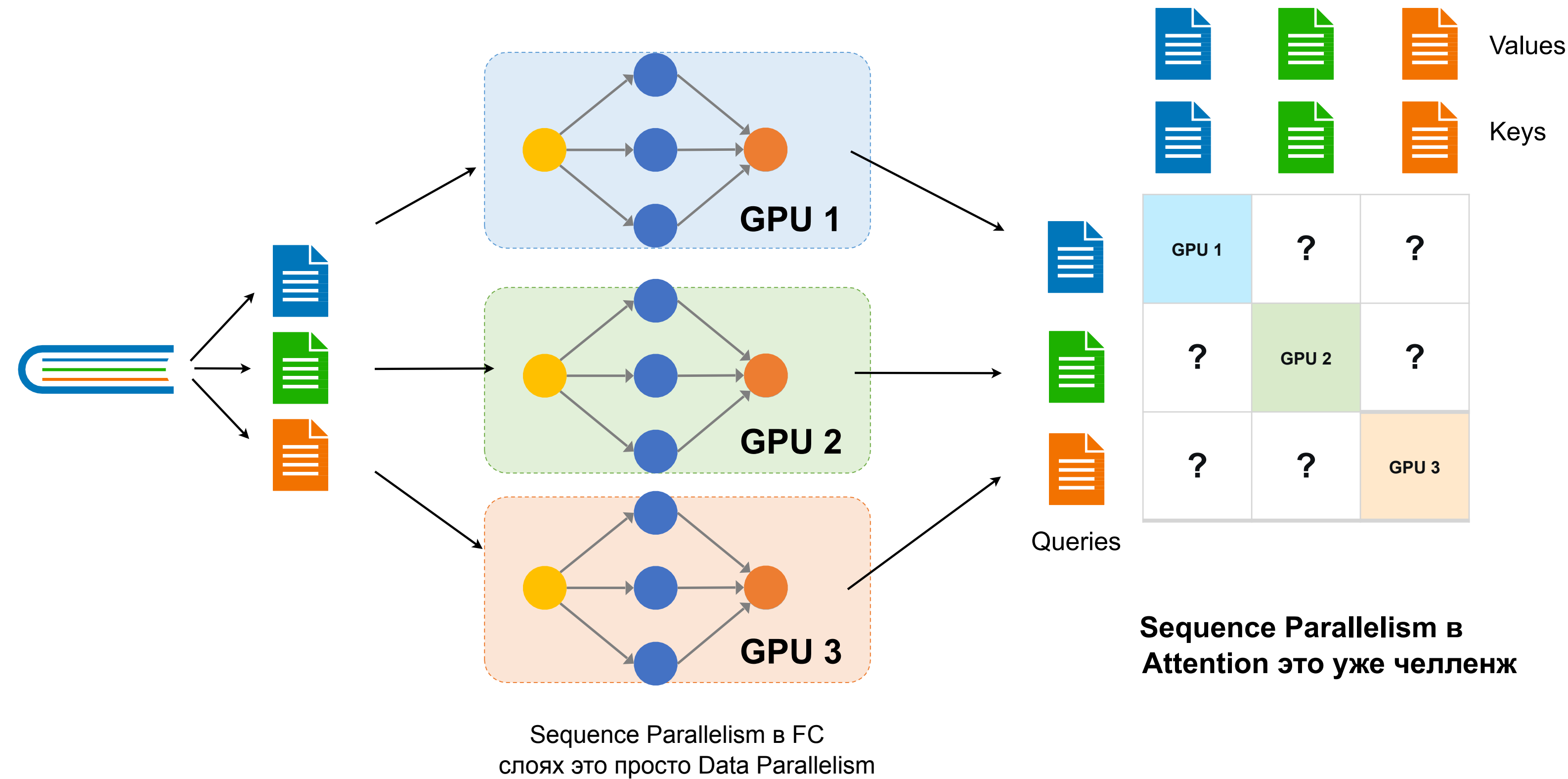
05

Offloading

ZeRO-Offload, ZeRO-Infinity



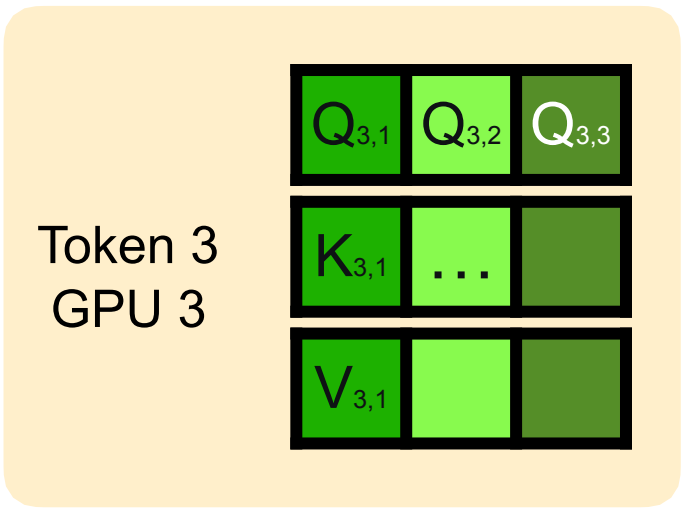
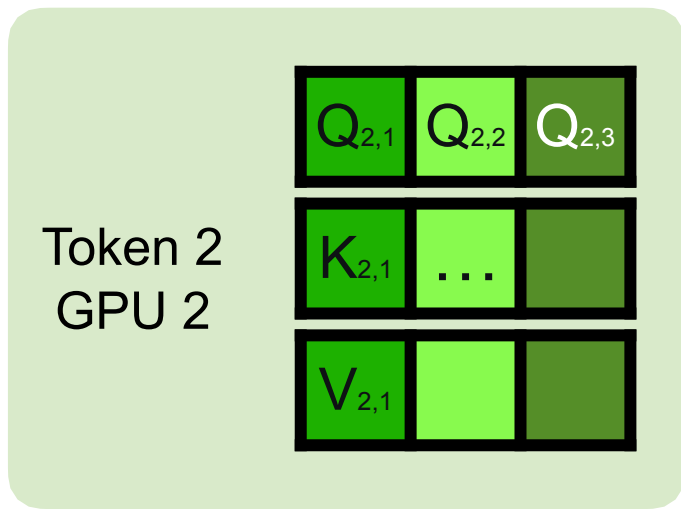
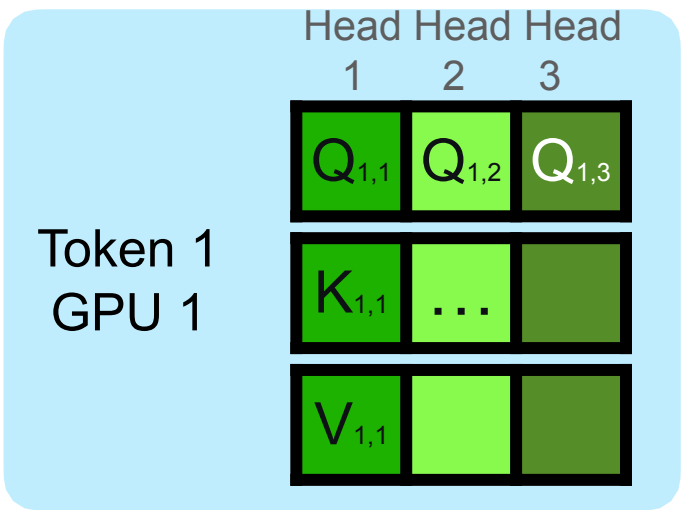
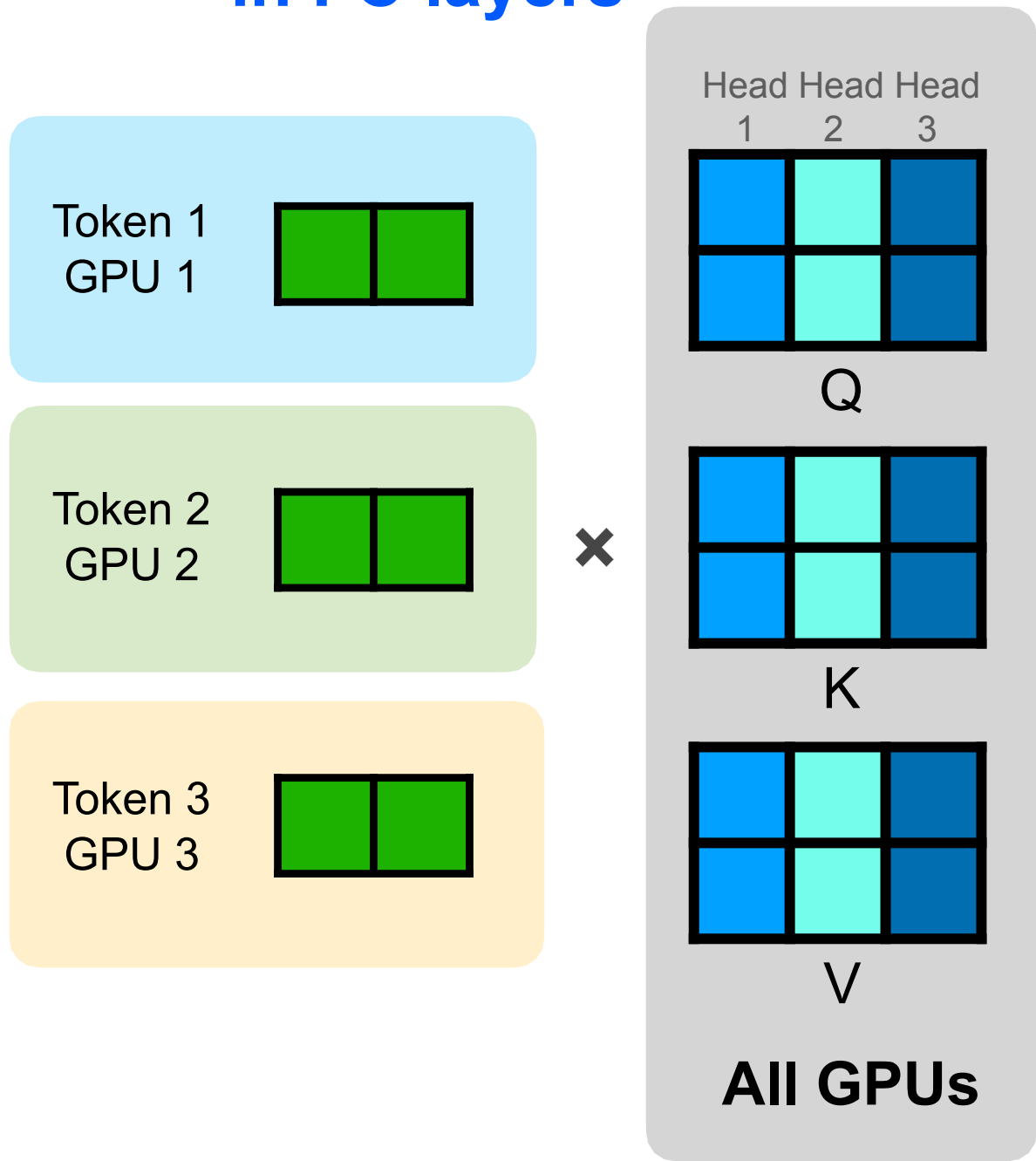
Sequence Parallelism



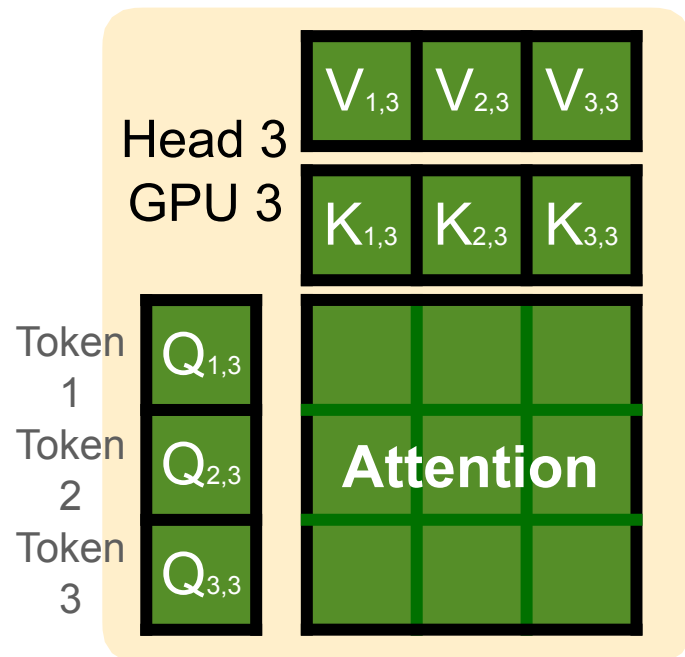
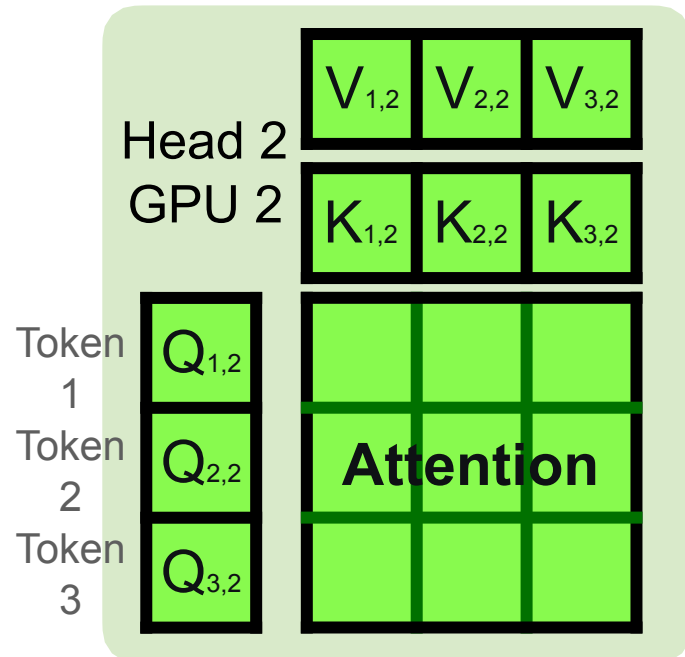
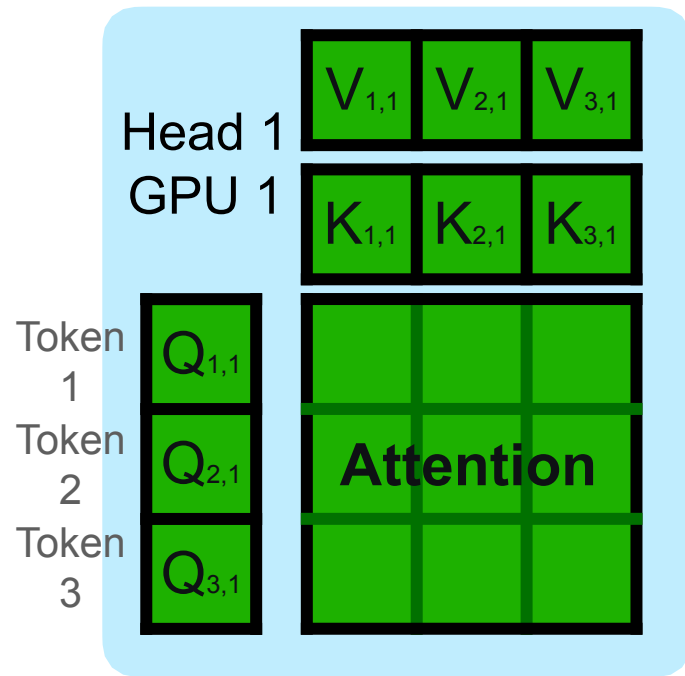
Sequence Parallelism

Решение 1: Re-partitioning

Split tokens
in FC layers



All to All
Communication
=>



Split Heads
in Attention
layers

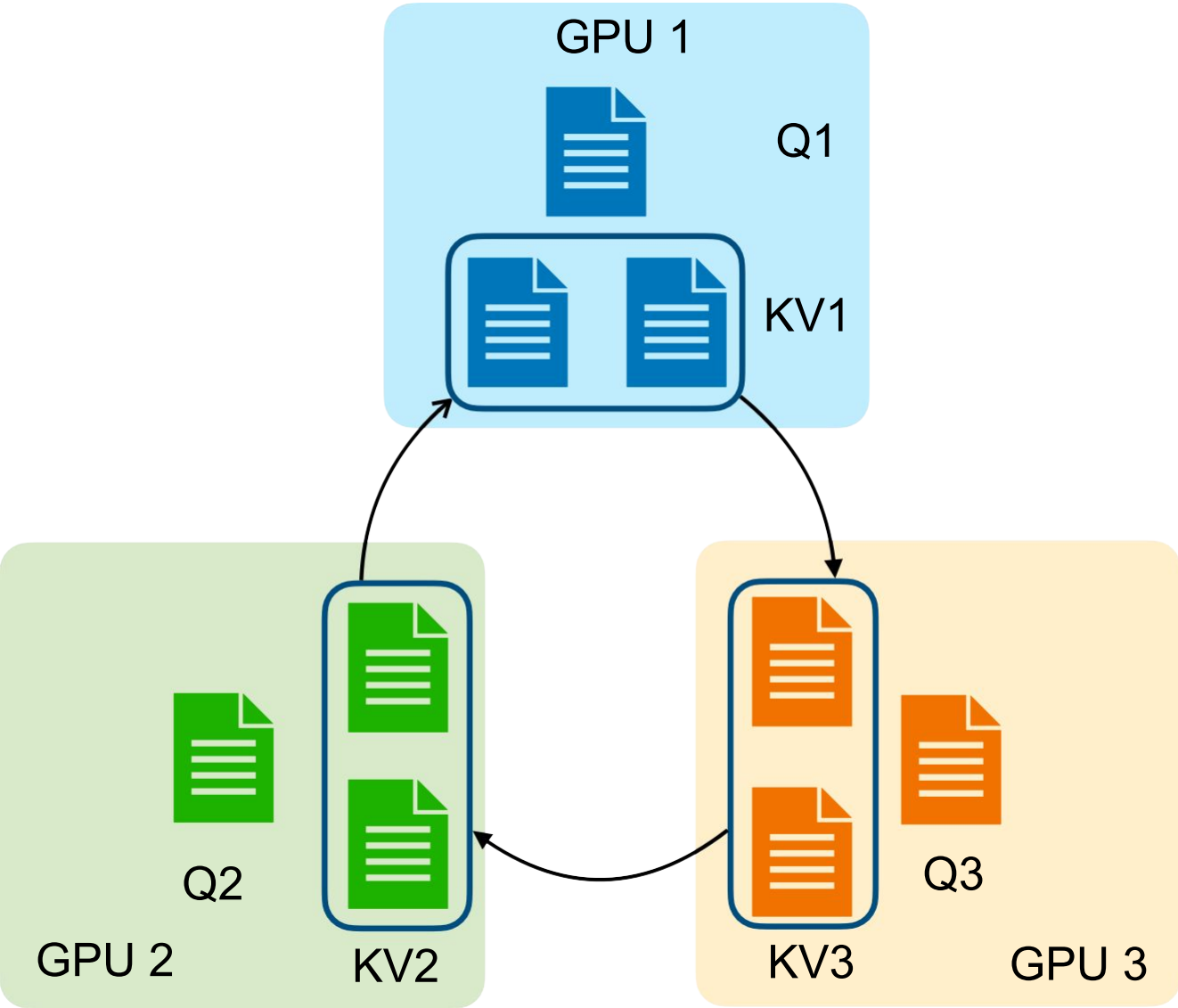
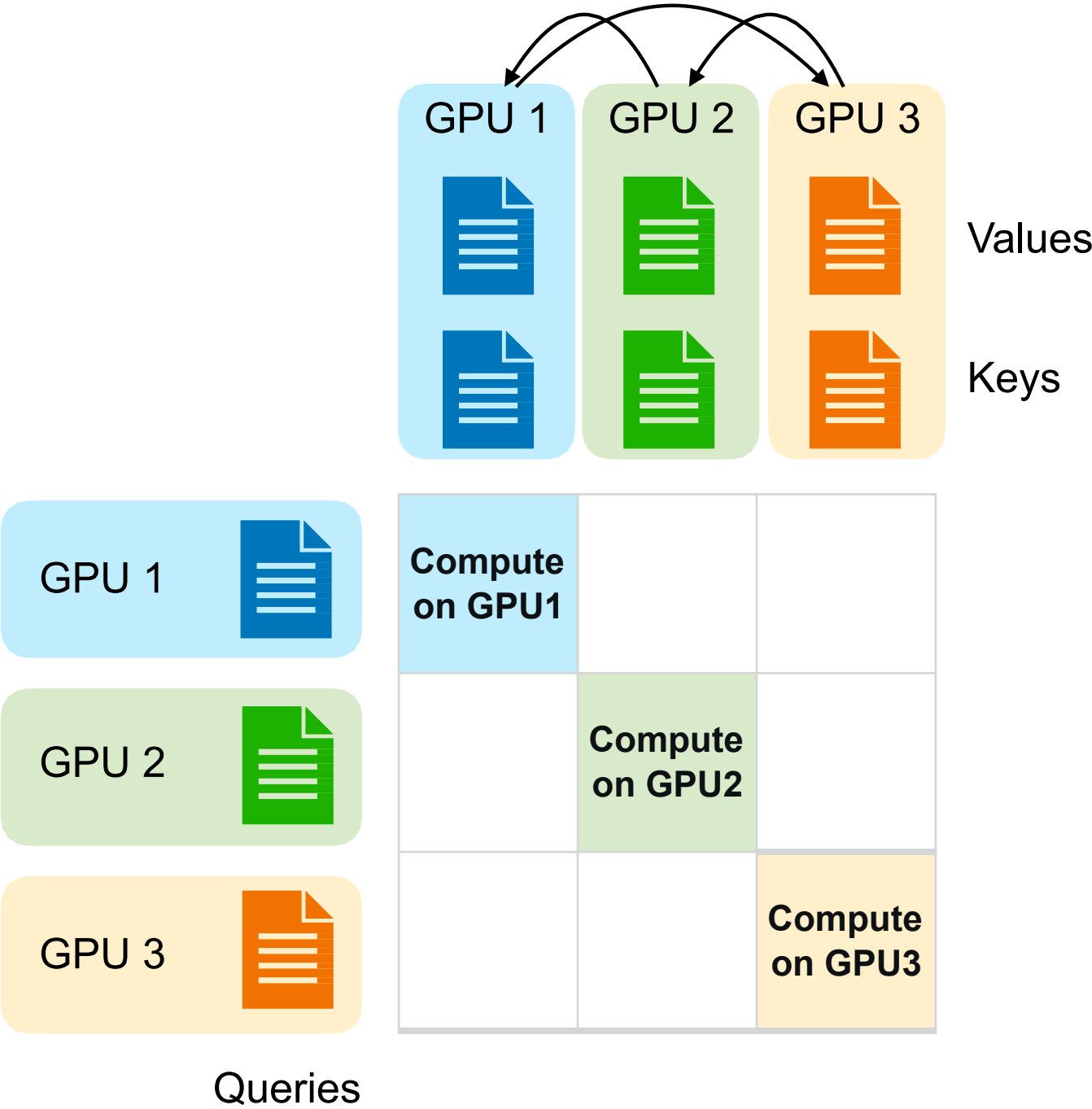
#GPUs
=
#Partitions of
Sequences
=
#Partitions of
Heads

Жесткое
ограничение!

Sequence Parallelism

Решение 2: Ring Attention

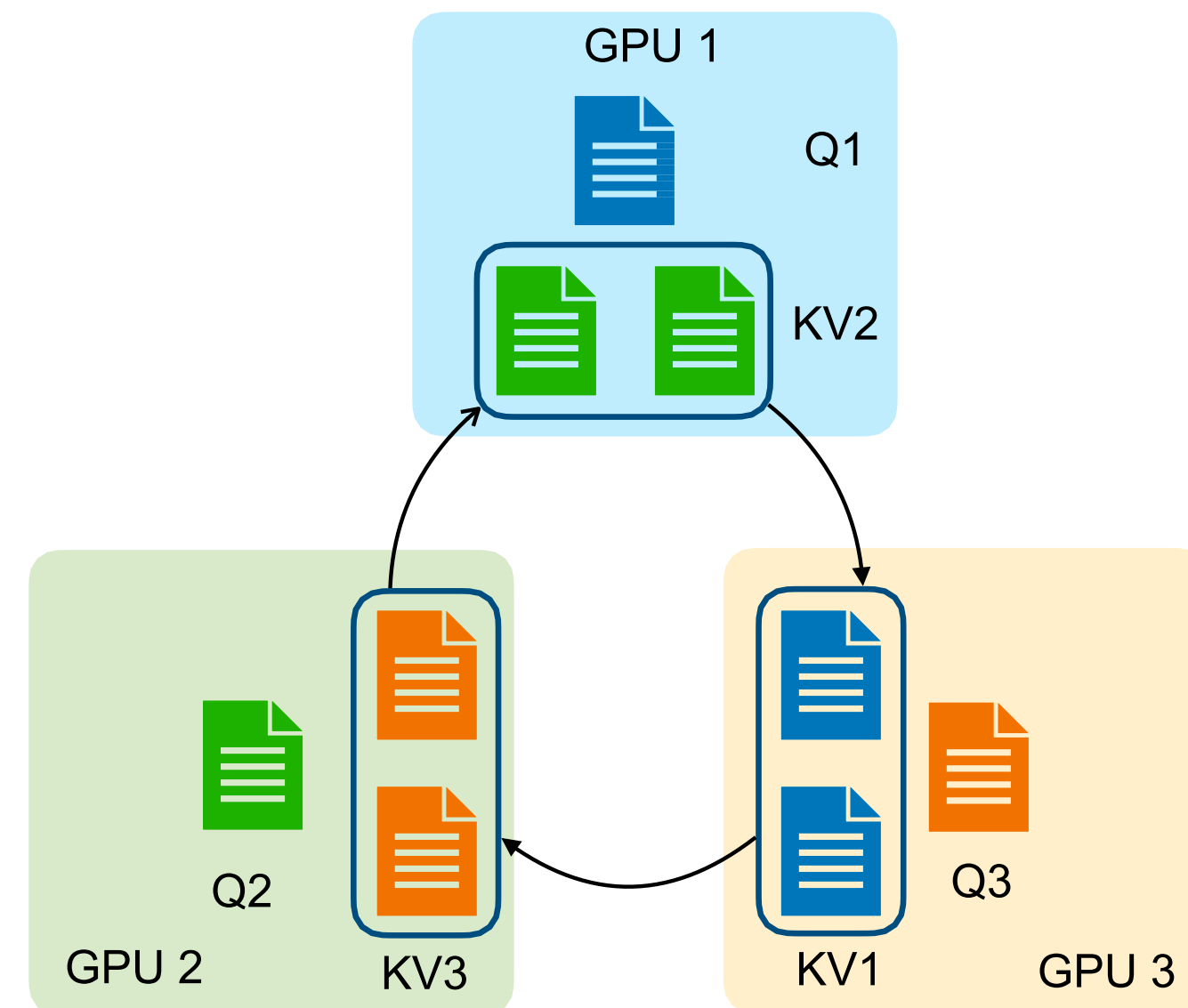
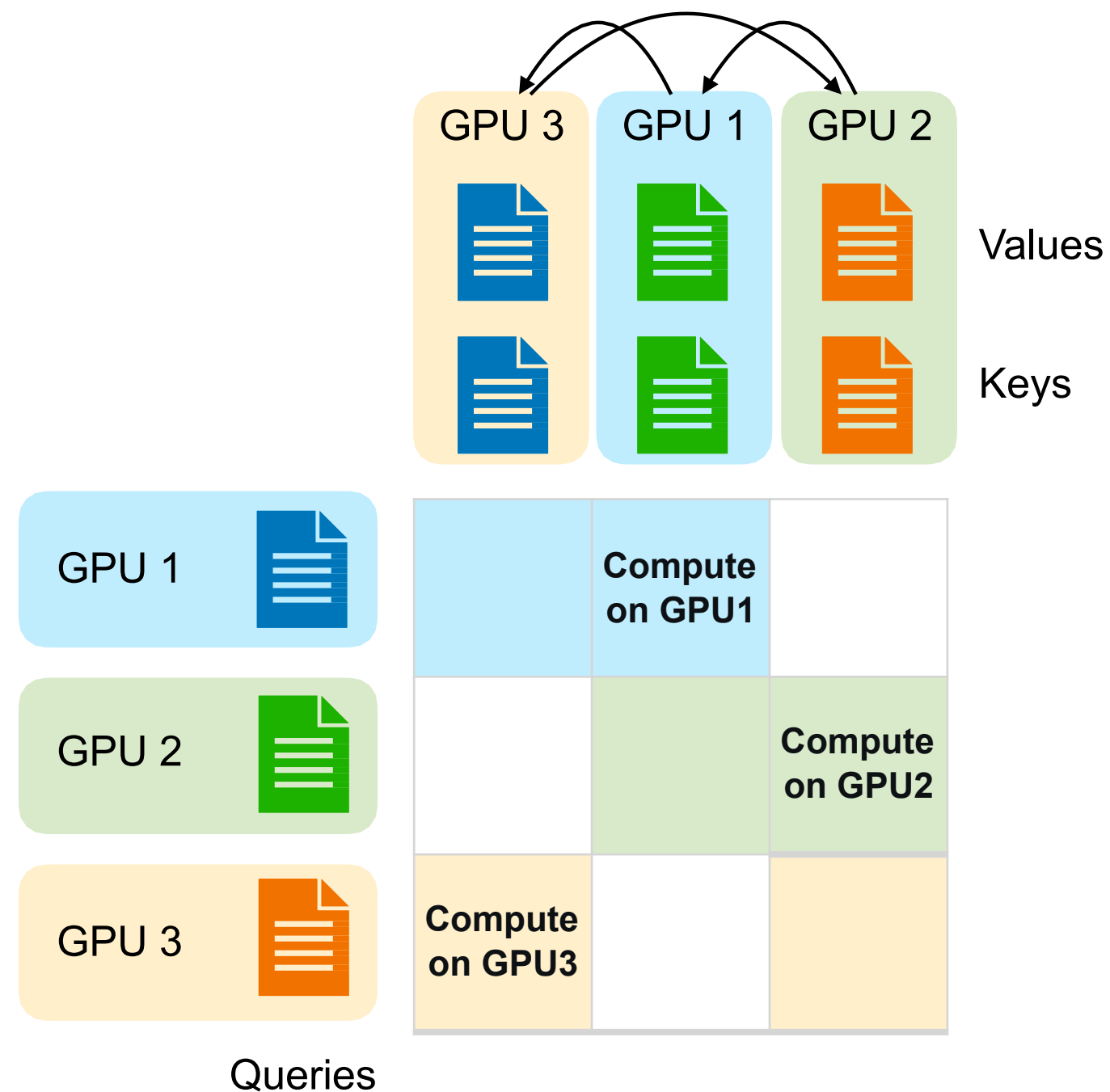
Пересылаем Keys и Values



Sequence Parallelism

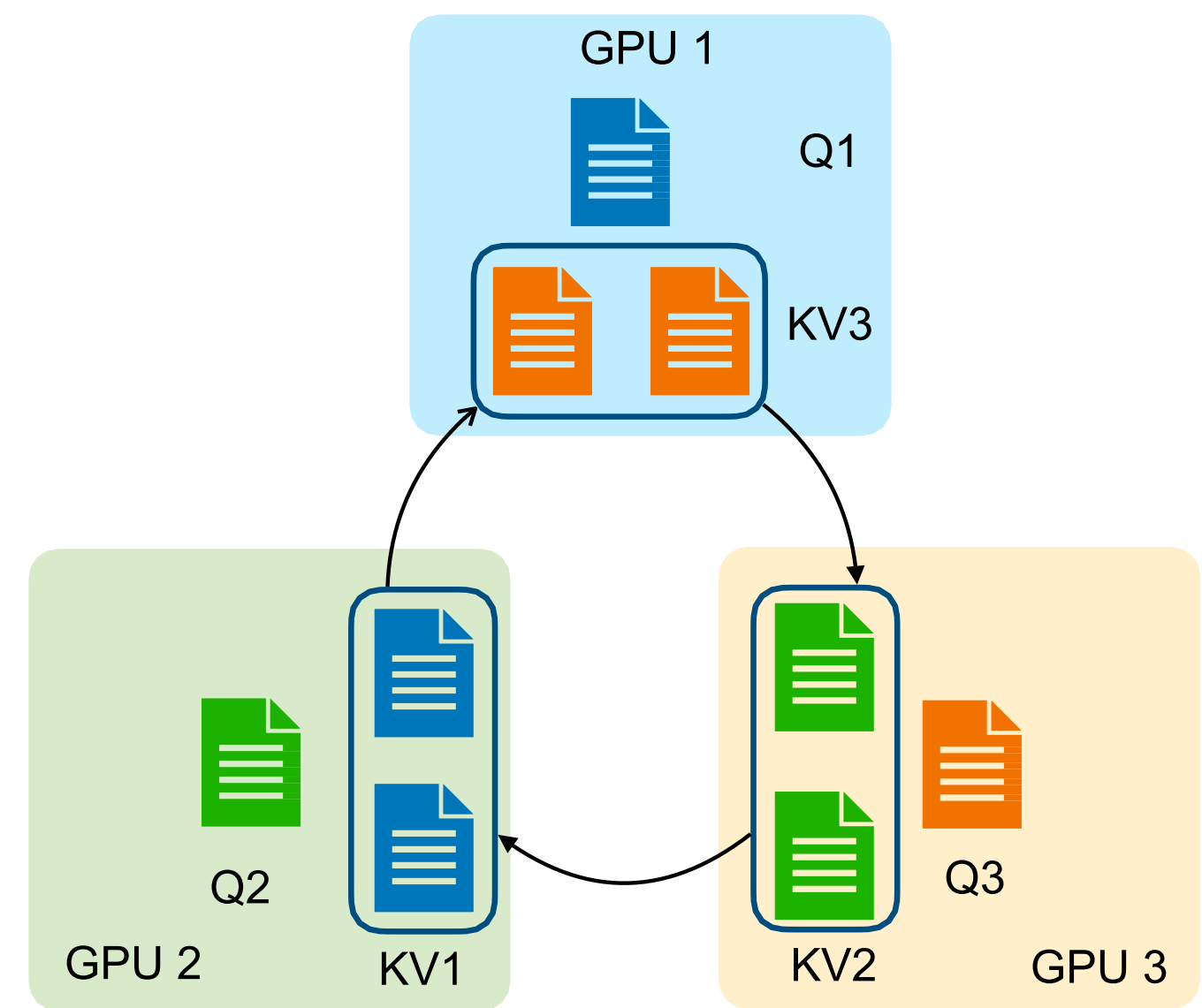
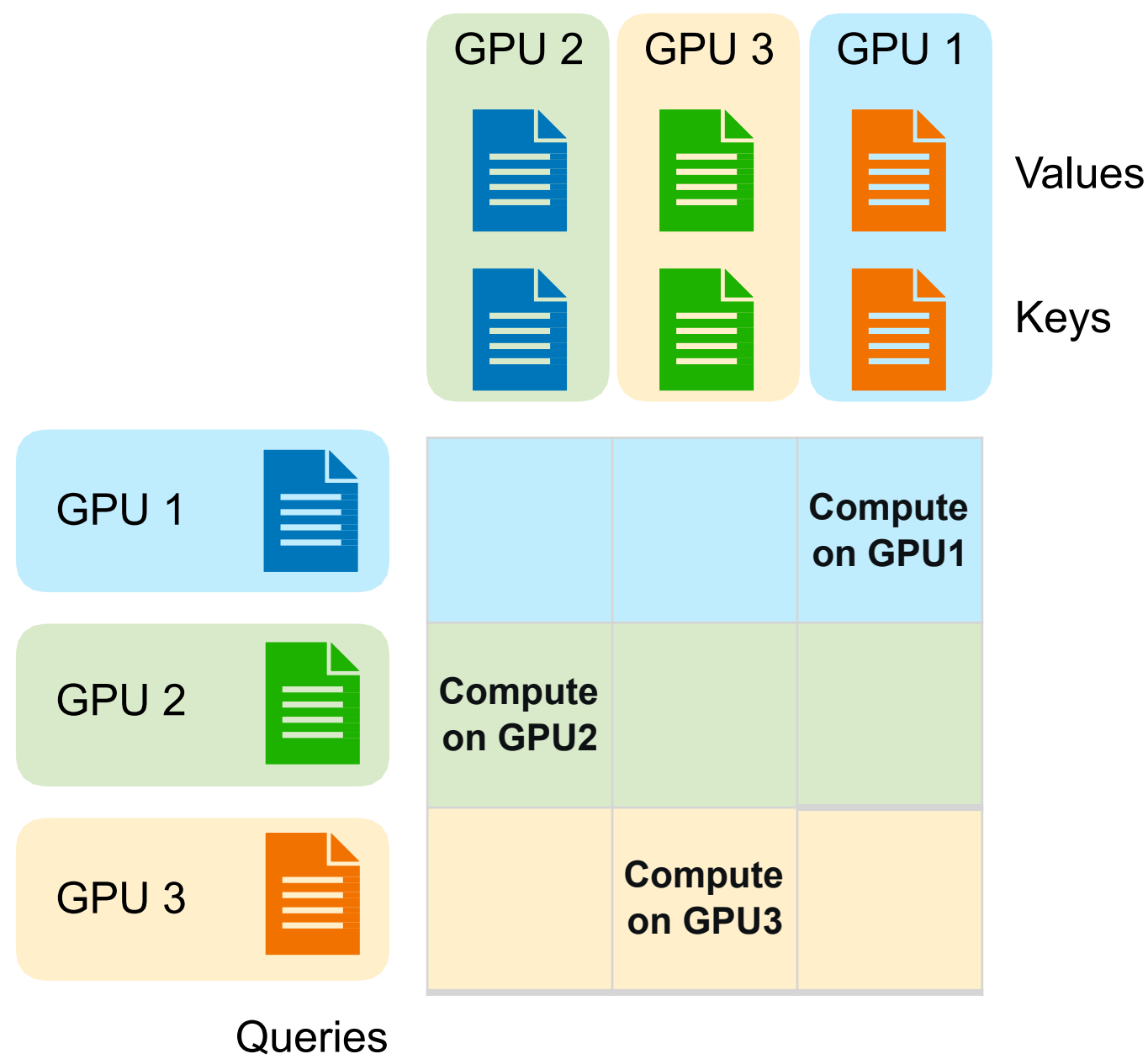
Решение 2: Ring Attention

Пересылаем Keys и Values



Sequence Parallelism

Решение 2: Ring Attention



План лекции

01

Pipeline Parallelism

Naive & Micro-Batch



02

Tensor Parallelism

На примере MLP & Self-Attention



03

Sequence Parallelism

Как считать Self-Attention?



04

Гибридный параллелизм

2D, 3D, 5D



05

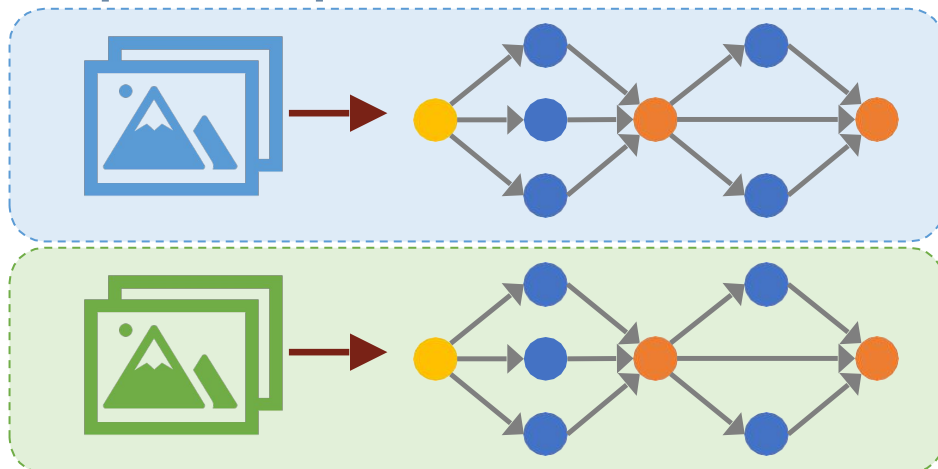
Offloading

ZeRO-Offload, ZeRO-Infinity



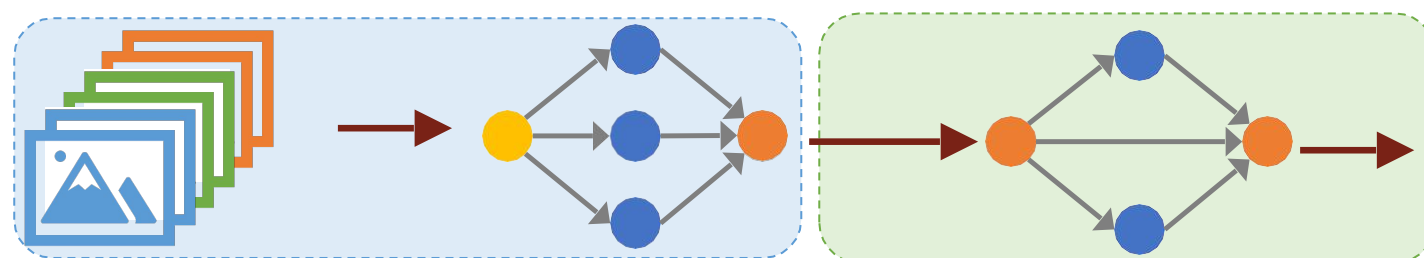
Гибридный параллелизм

Вспомним разобранные методы



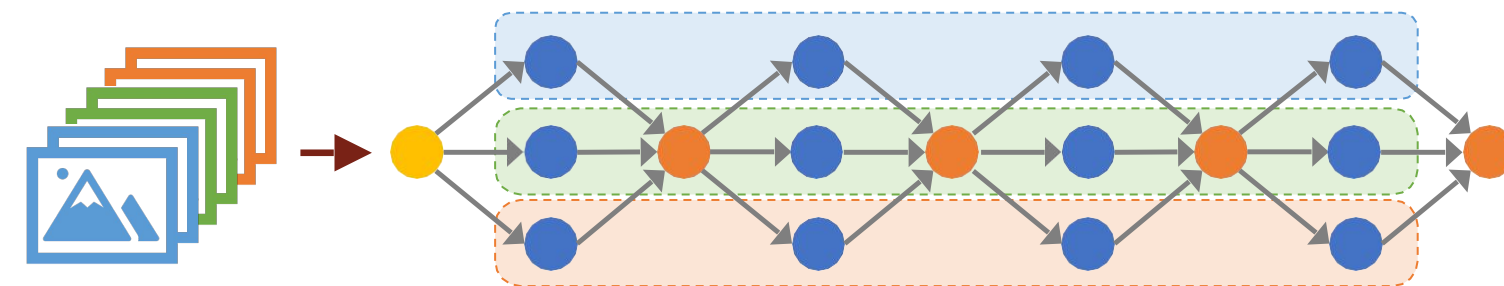
1. Data Parallelism:

- Разделяем данные
- N копий модели
- Высокая утилизация, высокий расход VRAM, мало коммуникаций
- Оптимизации: ZeRO-1,2,3, FSDP



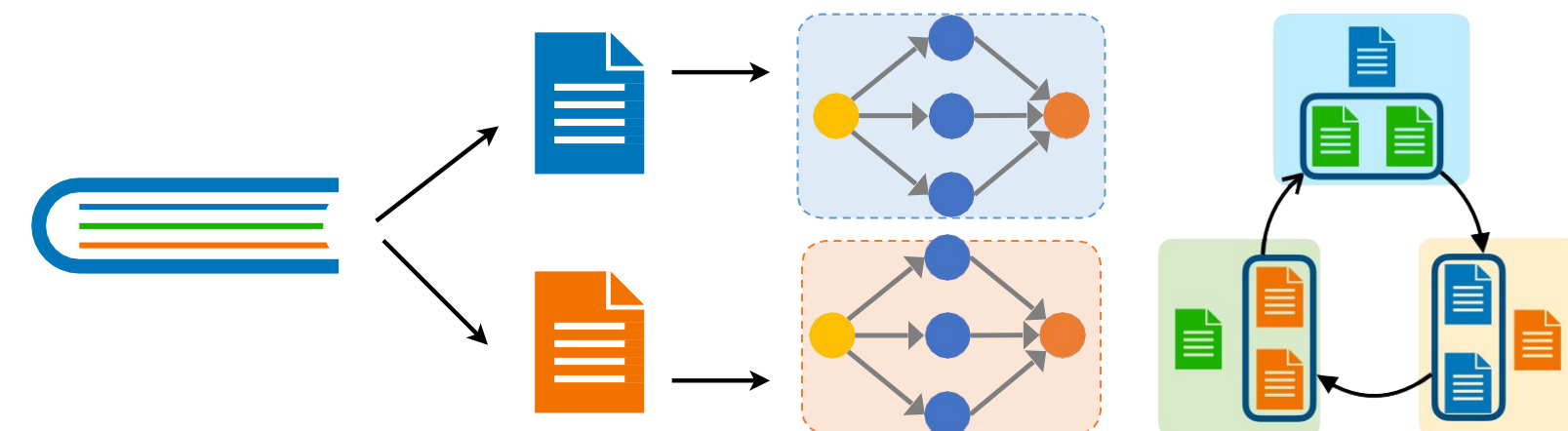
2. Pipeline Parallelism:

- Разделяем модель (слои)
- Одна копия модели
- Низкая утилизация, низкий расход VRAM, среднее кол-во коммуникаций



3. Tensor Parallelism:

- Разделяем модель (тензоры)
- Одна копия модели
- Высокая утилизация, низкий расход VRAM, много коммуникаций



4. Sequence Parallelism:

- Разделяем данные по токенам
- Доп. коммуникации в слоях Attention

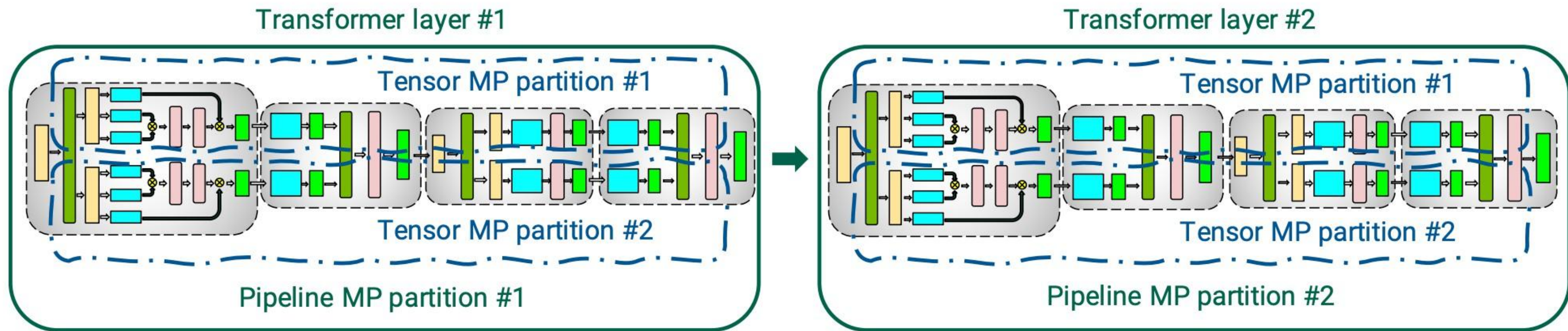
2D Parallelism



- Метод: PipeDream-Flush (1F1B)
schedule

Гибридный параллелизм

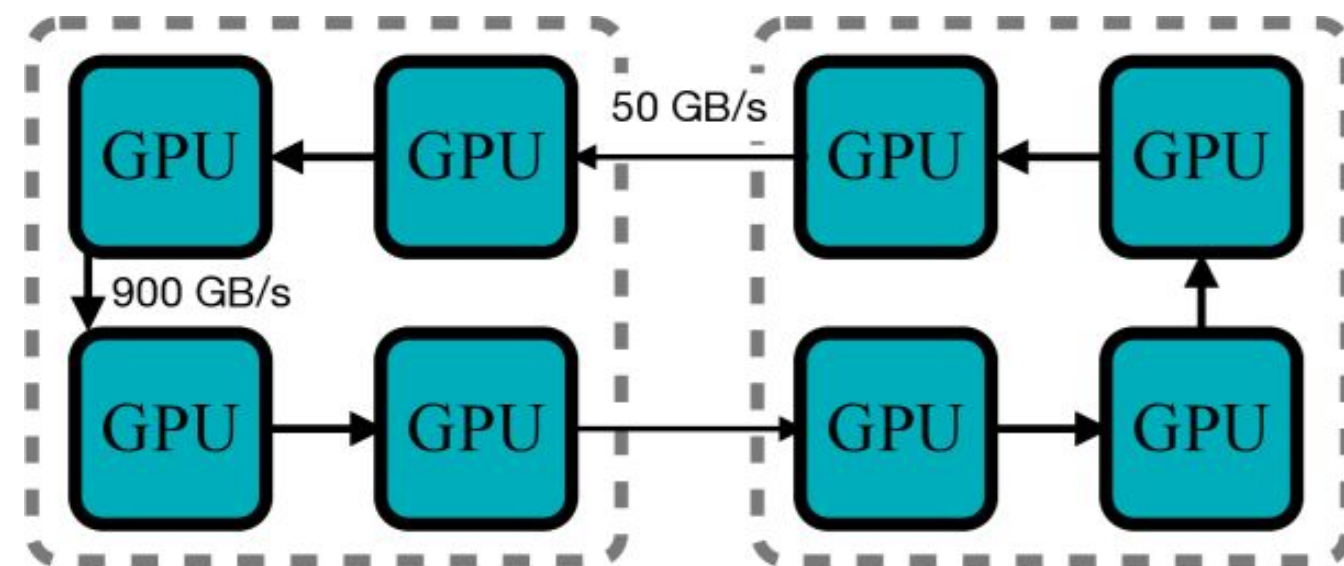
2D Parallelism



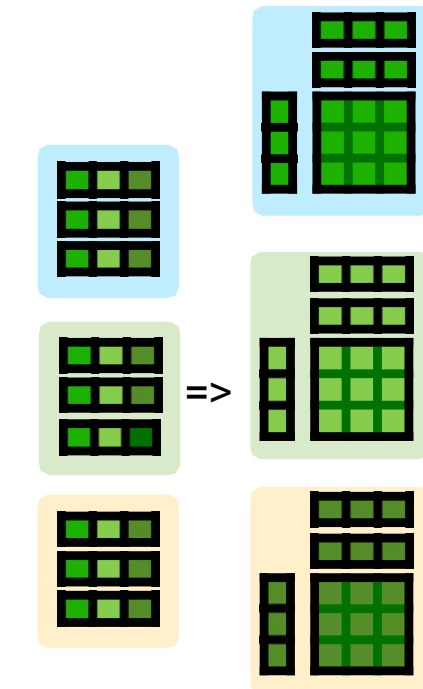
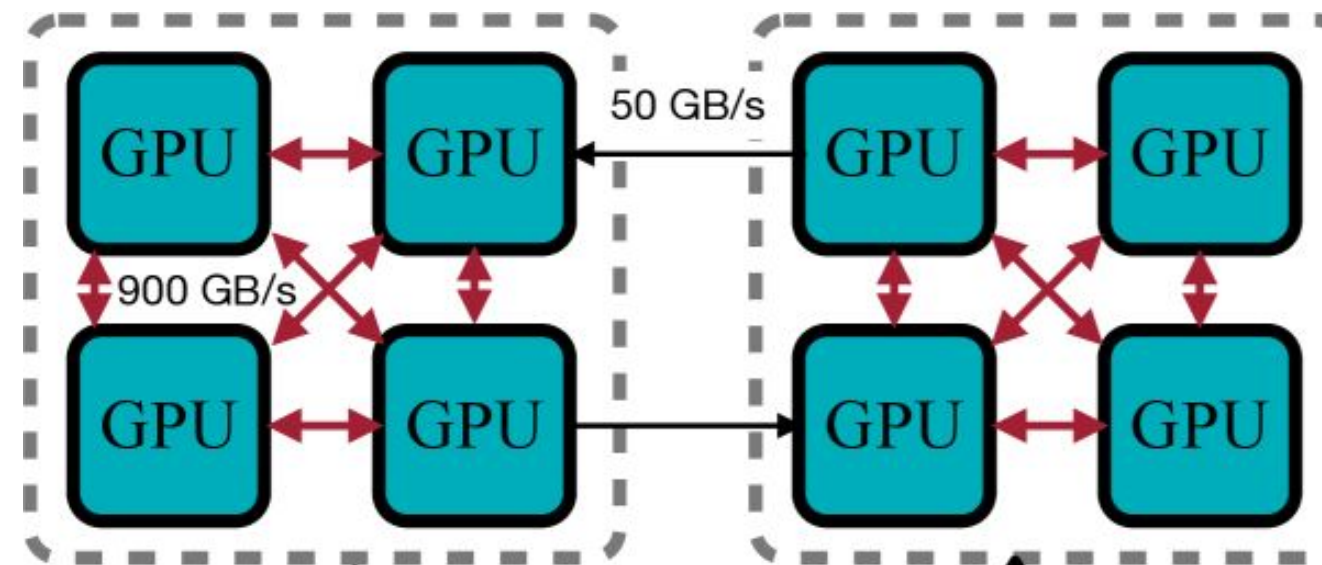
- Pipeline Parallelism + Tensor Parallelism
- “Снаружи”: Pipeline Parallelism
- “Внутри”: Tensor Parallelism

Гибридный параллелизм

2D Parallelism

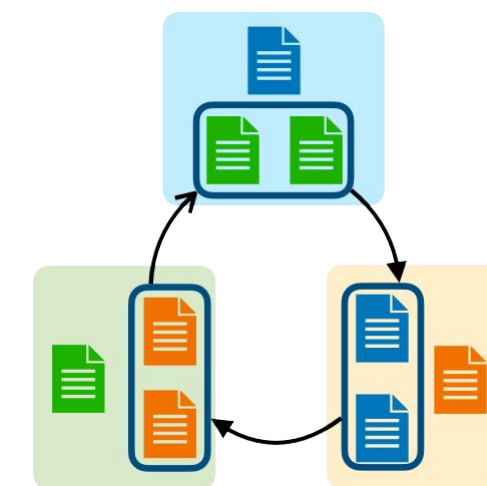


Недо-утилизация сети в рамках ноды



All-to-all re-partition в рамках ноды

- 2D Sequence Parallelism
- В рамках ноды: All-to-All re-partition
- Между нодами: Ring Attention

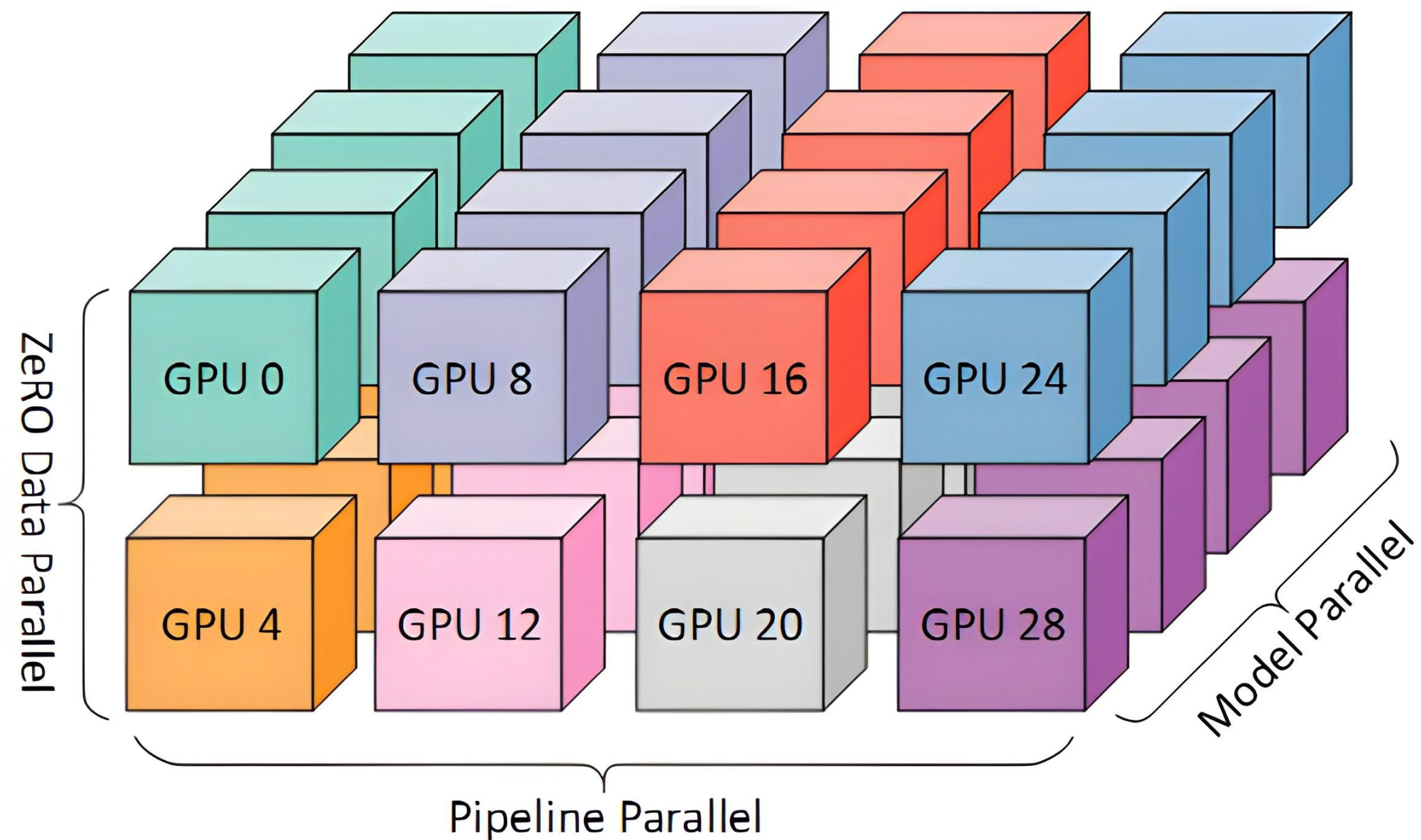


Ring Attention между нодами

2D Attention в LongVILA

Гибридный параллелизм

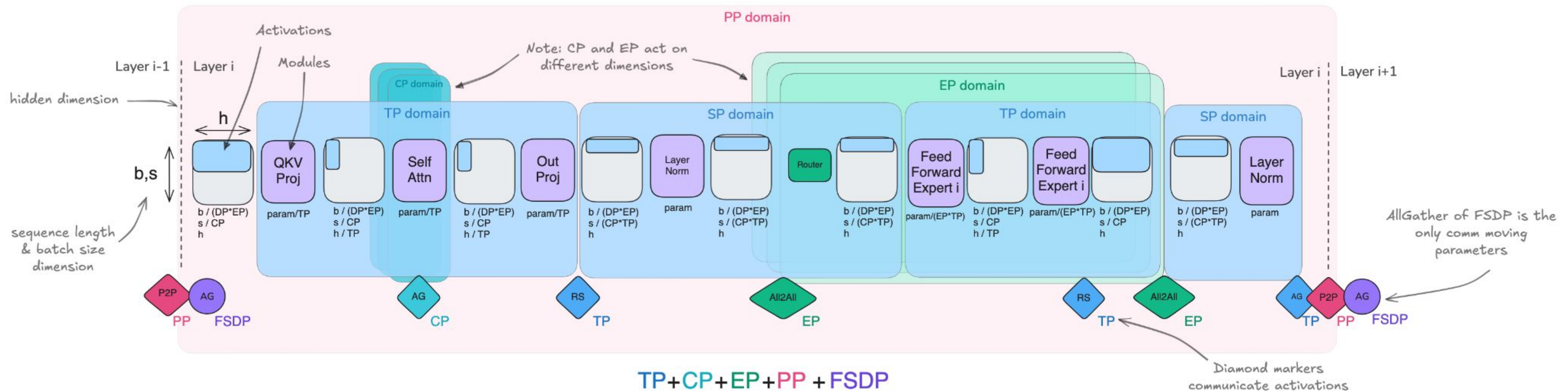
3D Parallelism



- Pipeline Parallelism + Tensor Parallelism + Data Parallel
- Одним цветом обозначены GPU на одном сервере

Гибридный параллелизм

5D Parallelism



- Применяем все что можем для одного слоя трансформера в МоЕ-сетапе! (в теории...)

Гибридный параллелизм

Скейлинг до 1Т параметров

- Применяли Pipeline Parallelism + Tensor Parallelism + Data Parallel
- Для полноценного претрейна GPT-3 в таком сетапе понадобится 84 дня

Model size	Hidden size	Number of layers	Model-parallel size	Number of GPUs	Batch size	Achieved teraFLOPs per GPU
1.7B	2304	24	1	32	512	137
3.6B	3072	30	2	64	512	138
7.5B	4096	36	4	128	512	142
18B	6144	40	8	256	1024	135
39B	8192	48	16	512	1536	138
76B	10240	60	32	1024	1792	140
145B	12288	80	64	1536	2304	148
310B	16384	96	128	1920	2160	155
530B	20480	105	280	2520	2520	163
1T	25600	128	512	3072	3072	163

Table 1. Weak-scaling throughput for GPT-3 models ranging from 1 billion to 1 trillion parameters.

Гибридный параллелизм

Сравнение различных методов

Method	Memory Savings Focus	Parallel/Sharding Dimension	Primary Disadvantage
DP	Activations (via smaller microbatch)	Batch	Memory Redundancy per device
PP	Model Parameters	Model Layers	Pipeline Bubble / Sched. Complexity
TP/SP	Params and Activations	Hidden Dim / Sequence Length	Requires High Intra-Node Bandwidth
CP	Activations (for long sequences)	Sequence Length	Attention Comm. Overhead (Ring)
EP	Expert Parameters	Expert Dimension	Requires MoE / All-to-All Overhead
ZeRO-1	Optimizer States	Sharded across DP Replicas	Added Param Comm. (AllGather)
ZeRO-2	Optimizer States, Gradients	Sharded across DP Replicas	Added Param Comm. (AllGather)
ZeRO-3	Params, Gradients, Optimizer States	Sharded across DP Replicas	Increased Param Comm. Volume (Many AllGathers)

Гибридный параллелизм

Это лишь вершина айсберга

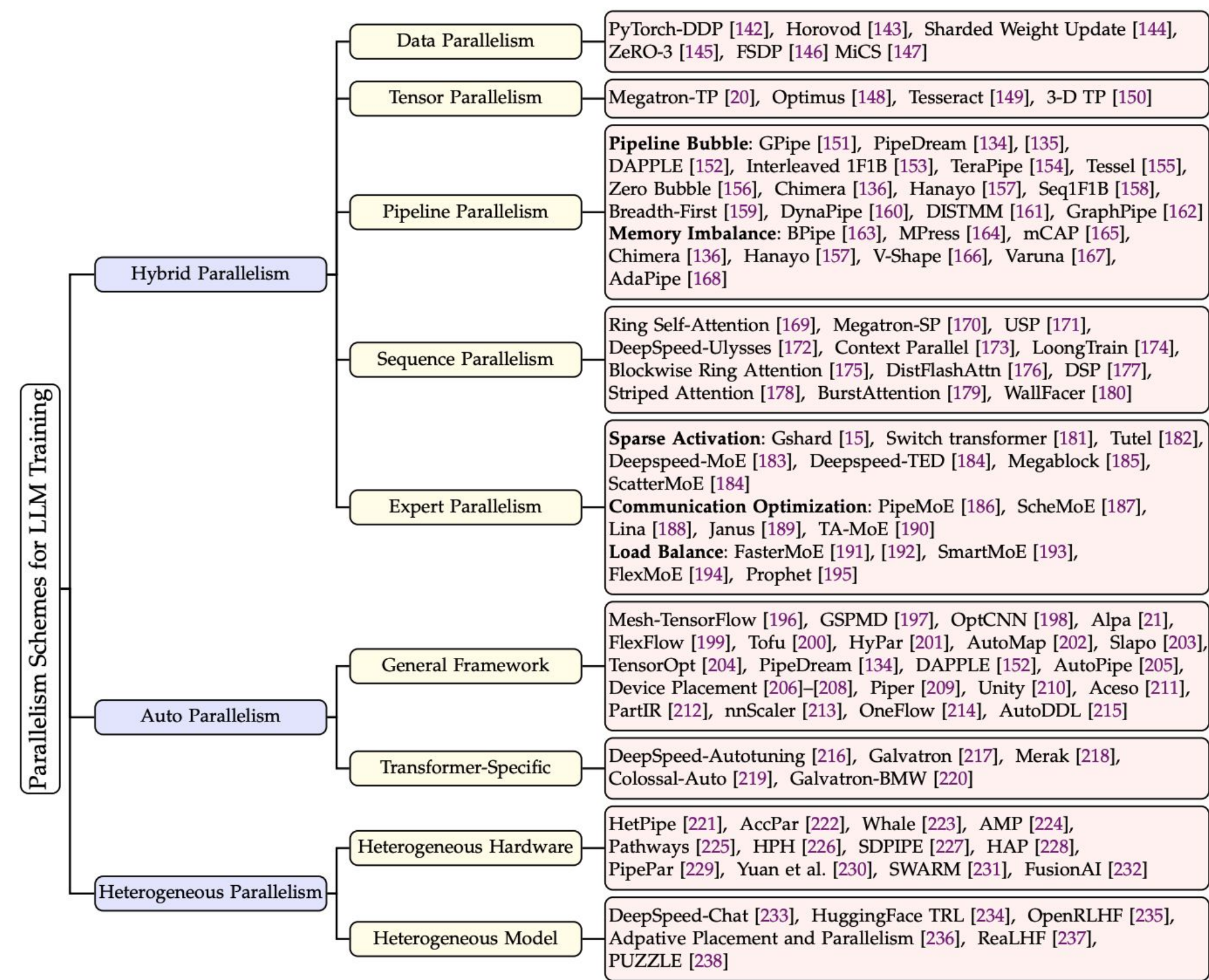


Fig. 7: Studies on parallelism schemes for distributed LLM training.

Гибридный параллелизм

Это лишь вершина айсберга

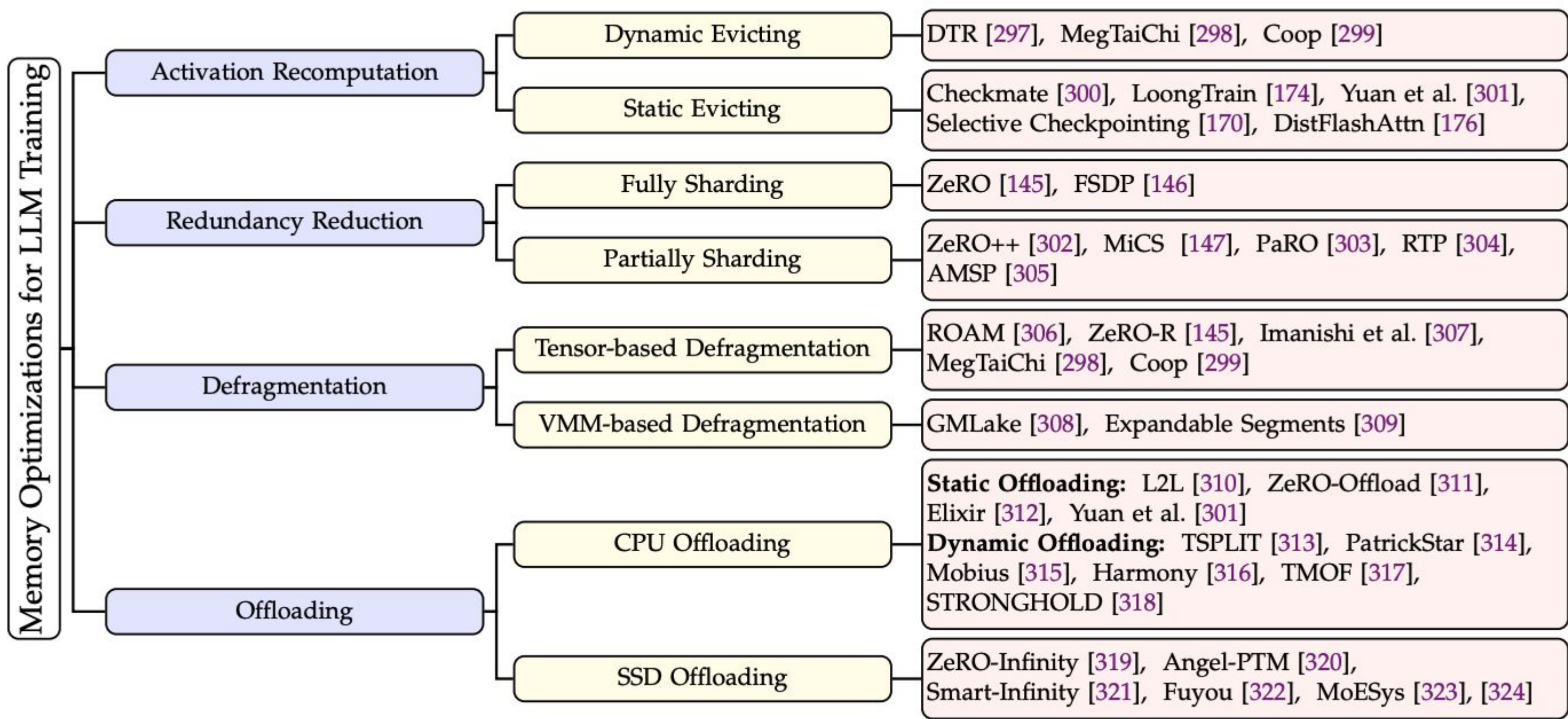


Fig. 12: Studies on memory optimizations for distributed LLM training.

План лекции

01

Pipeline Parallelism

Naive & Micro-Batch



02

Tensor Parallelism

На примере MLP & Self-Attention



03

Sequence Parallelism

Как считать Self-Attention?



04

Гибридный параллелизм

2D, 3D, 5D



05

Offloading

ZeRO-Infinity



Offloading

ZeRO-Infinity

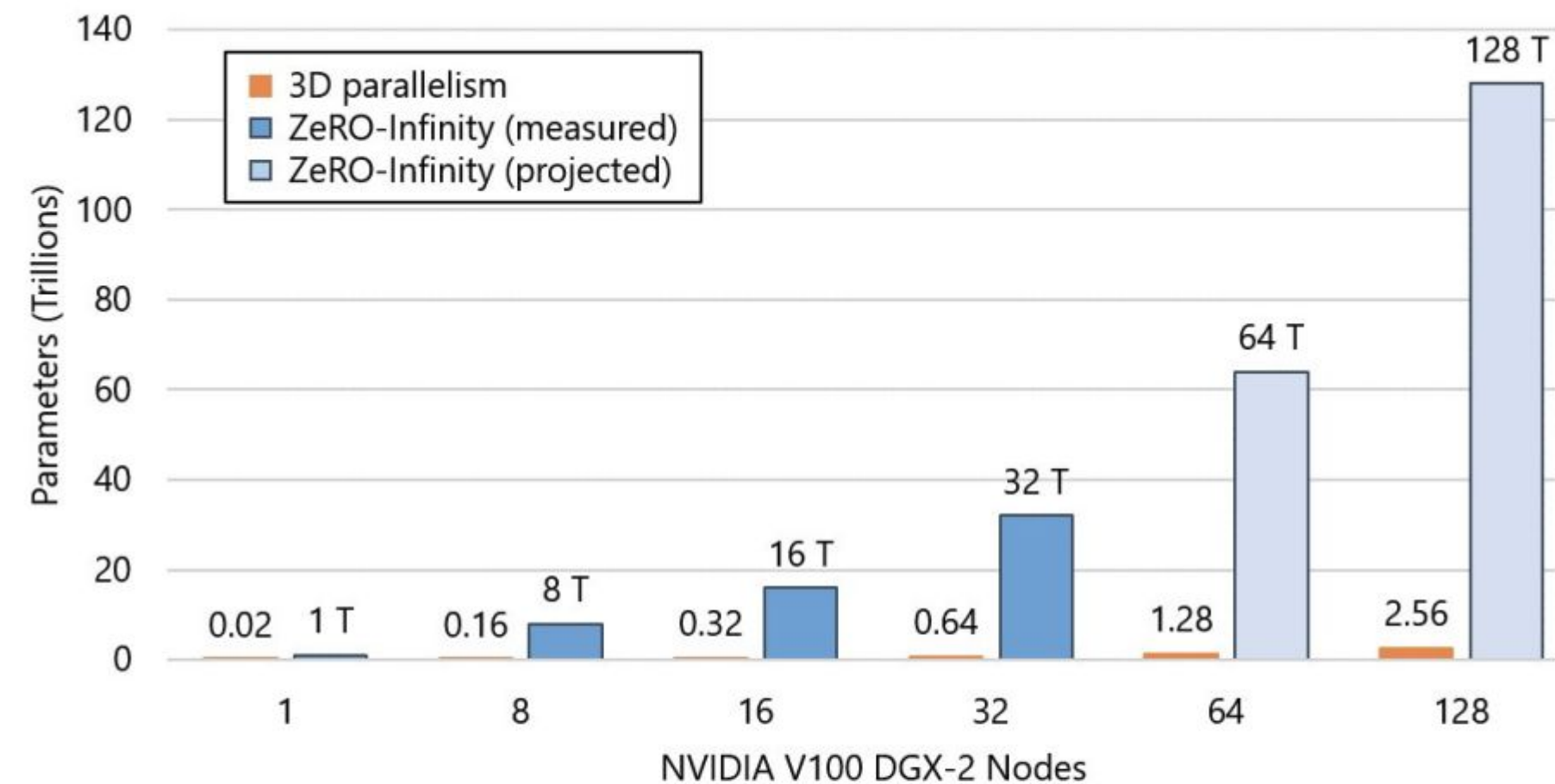


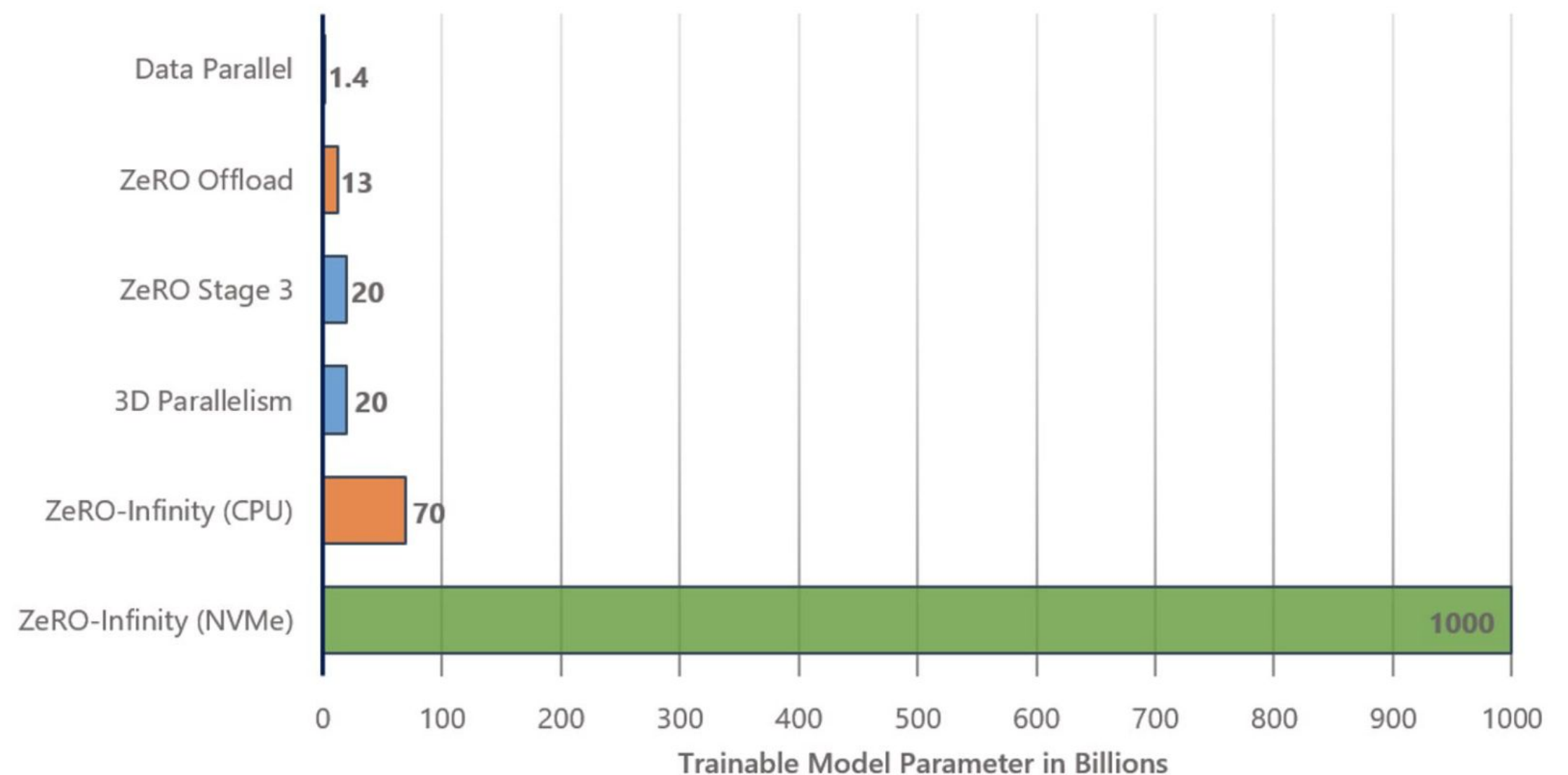
Figure 1: ZeRO-Infinity can train a model with 32 trillion parameters on 32 NVIDIA V100 DGX-2 nodes (512 GPUs), 50x larger than 3D parallelism, the existing state-of-the-art.

Offloading

ZeRO-Infinity

Как такое достигается?

- ZeRO-3 + сгружаем модель на CPU/NVMe
- Загружаем на GPU по требованию
- Много инженерных оптимизаций под капотом для перекрытия compute/communication



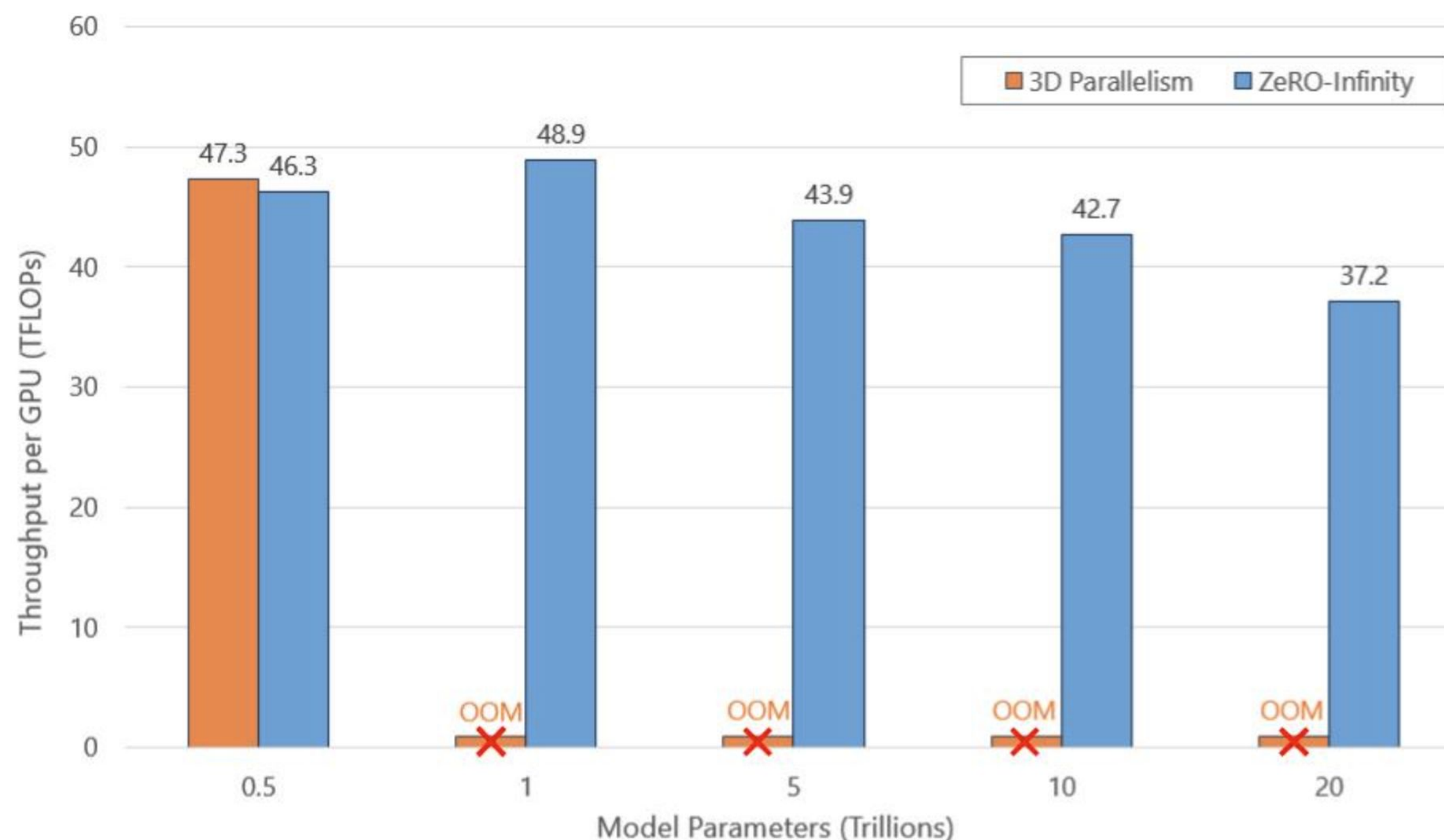
Offloading

ZeRO-Infinity

Еще и хорошо скейлится!

Какие же подводные?

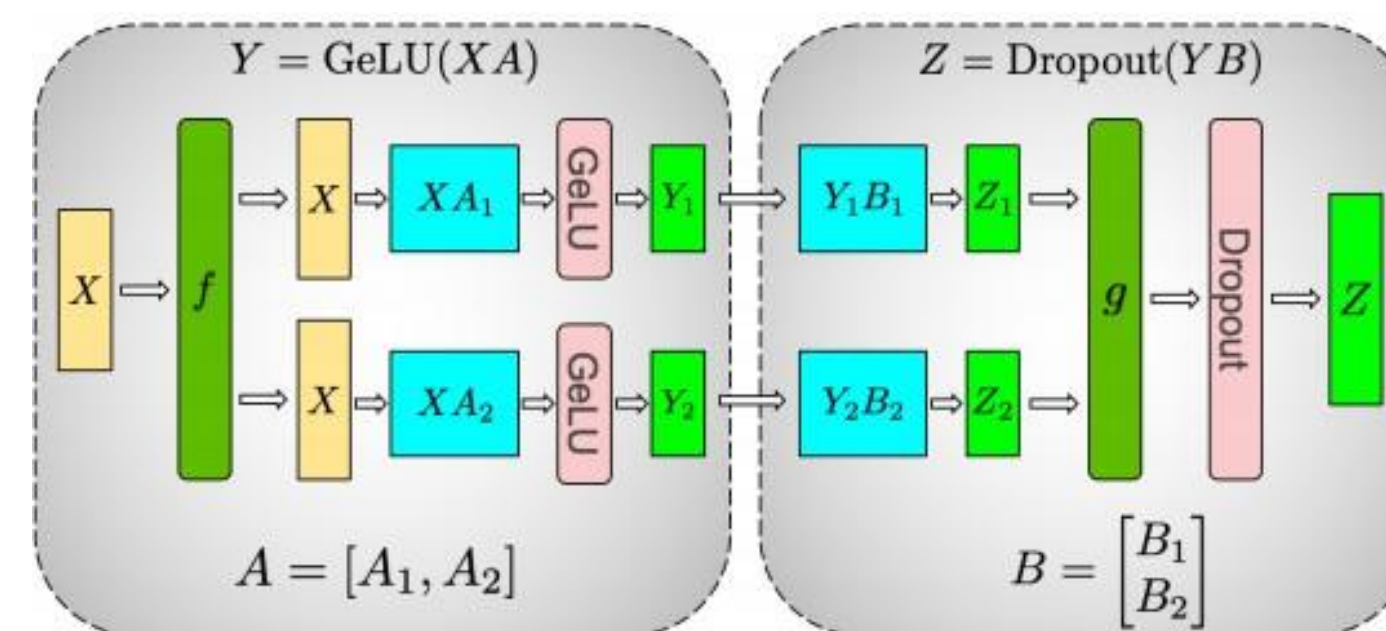
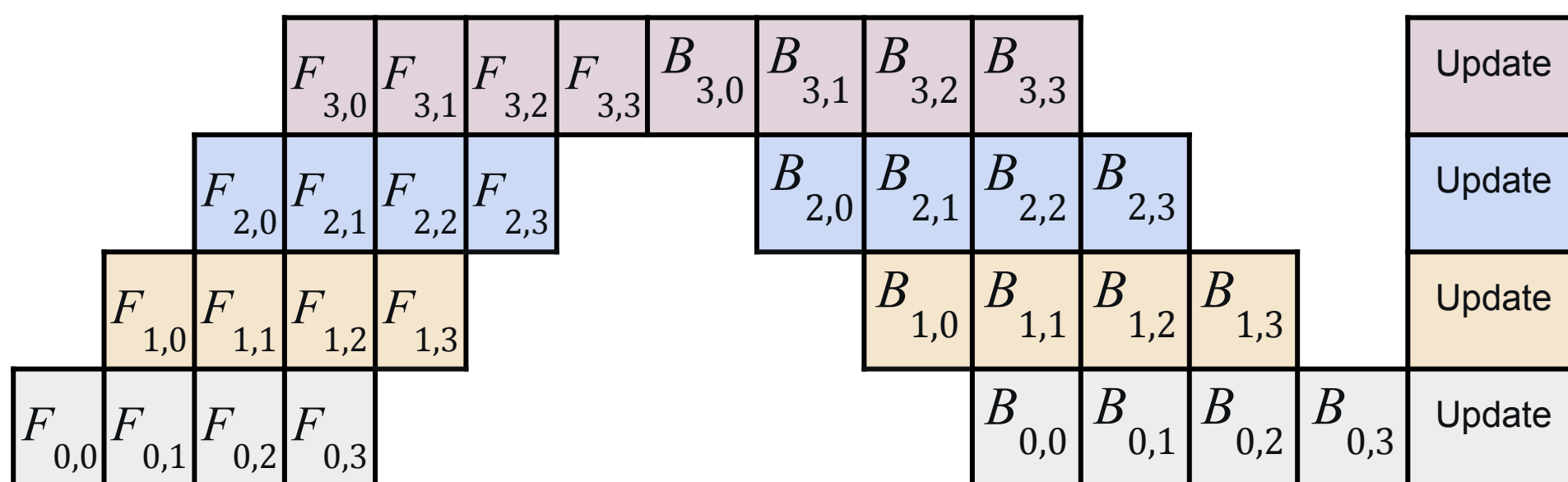
- Большие требования к пропускной способности. Малейшая просадка — перекрывать compute/communication не получится
- Это инструмент для пробития “GPU Memory Wall” для обучения *экстремально* больших моделей, когда обычный 3D уже не спасает



For optimizer states, the authors of the paper state that the communication cannot be overlapped with computation. Therefore, a bandwidth of **1.5TB/s** is required in order to achieve a 90% efficiency. <https://zhuogege1943.com/blogs/2025/01/13/ZeRO-Infinity/>

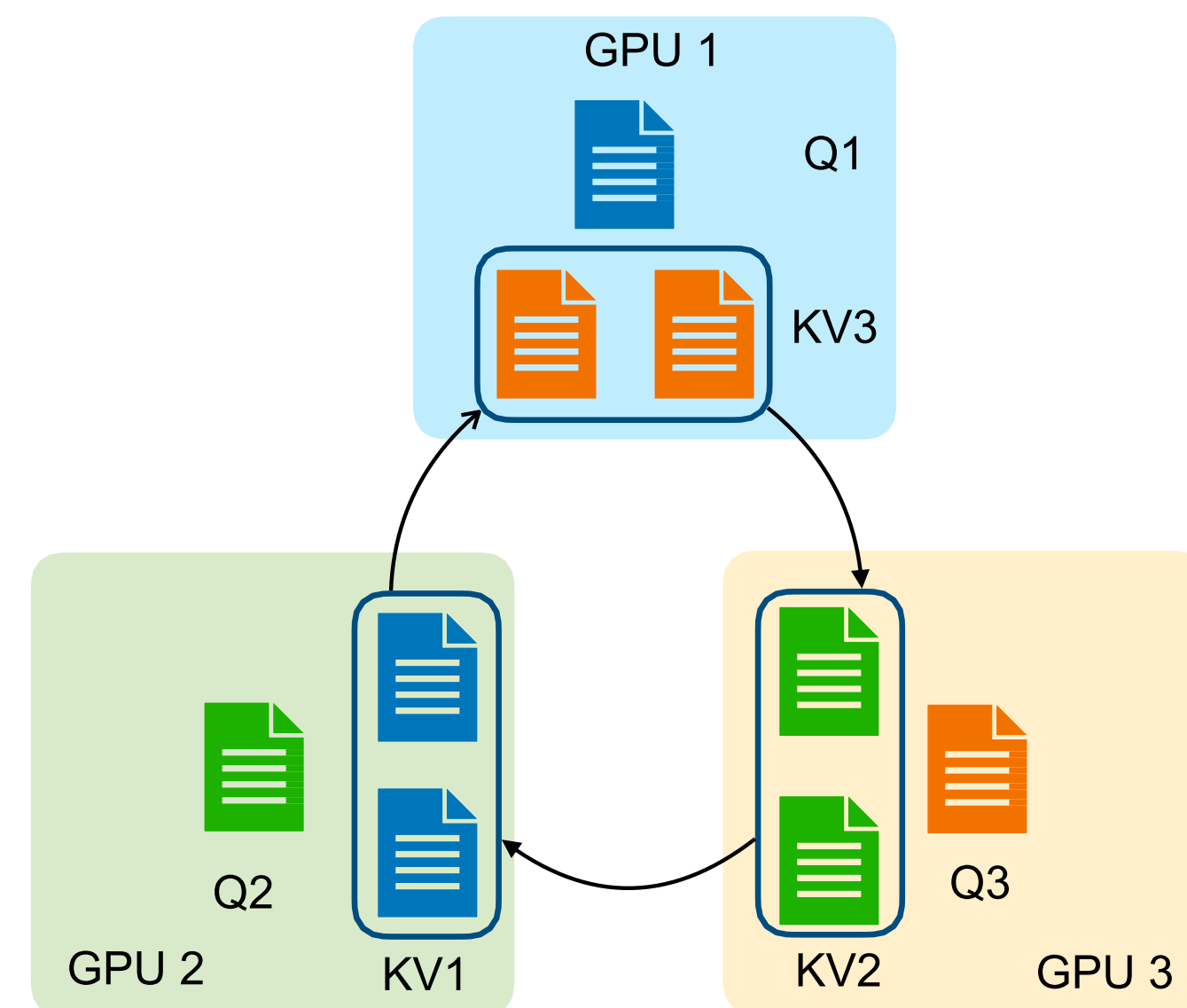
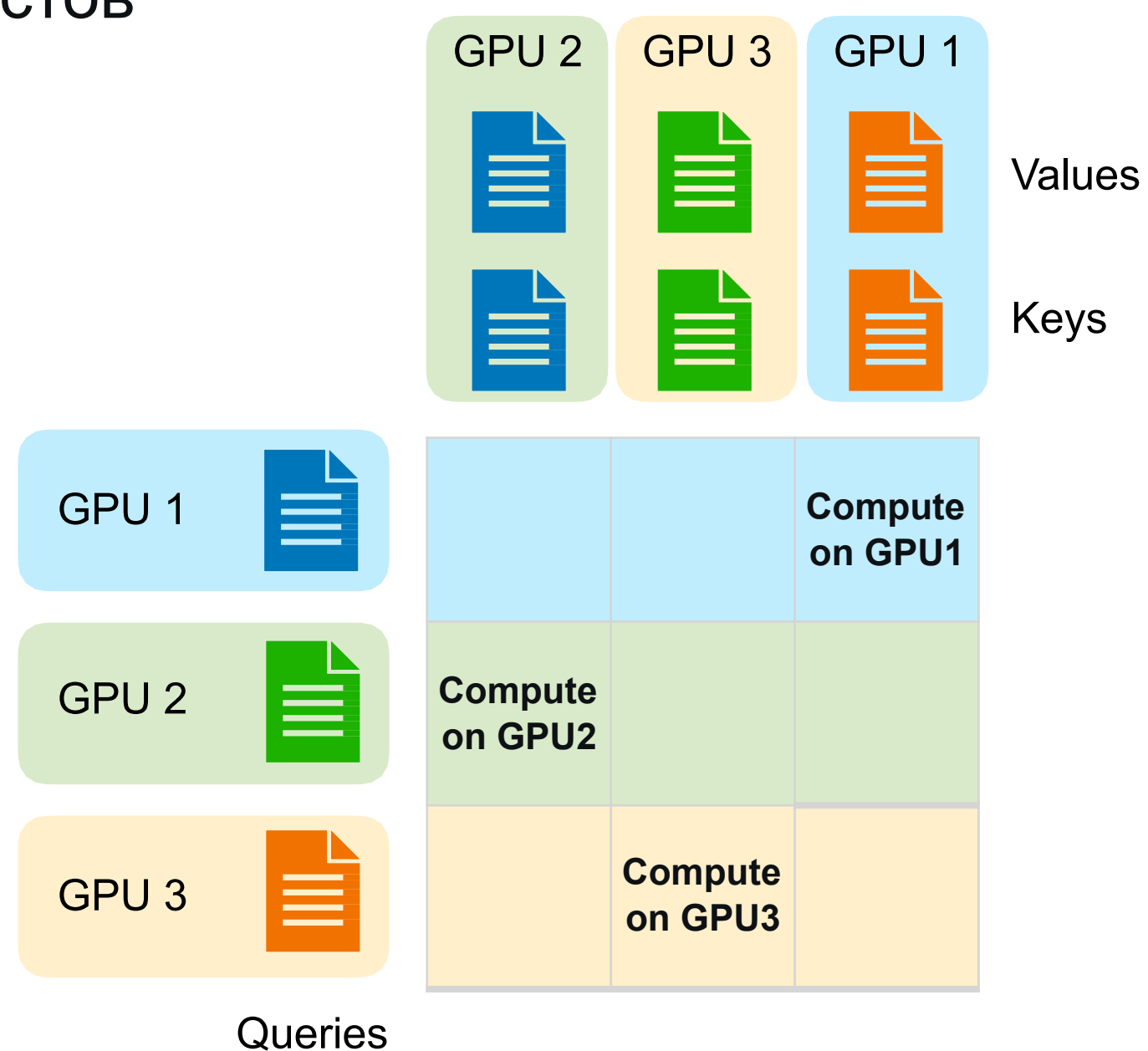
Резюме сегодняшней лекции

- Разобрали Pipeline & Tensor Parallelism (Model Parallelism)



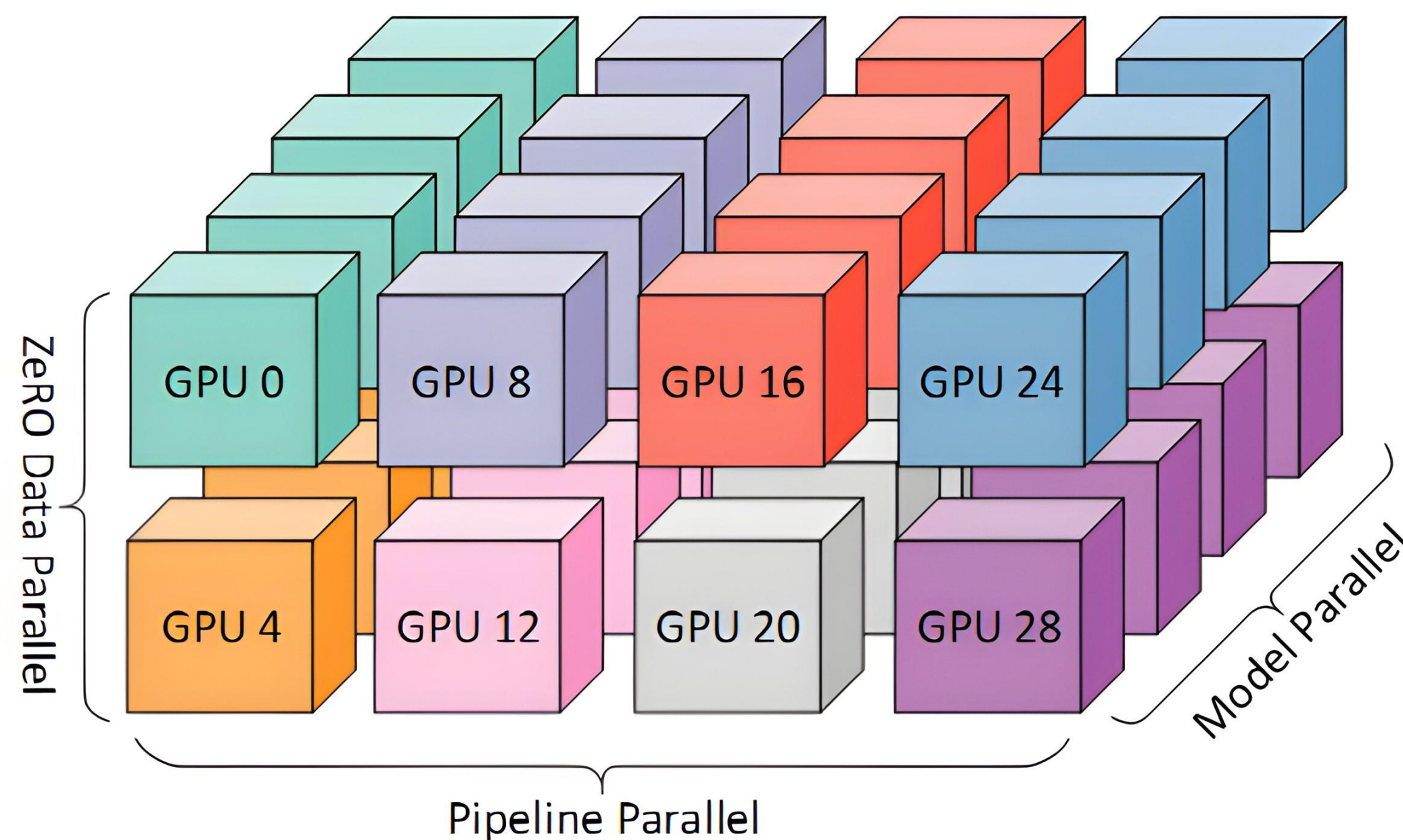
Резюме сегодняшней лекции

- Разобрали Pipeline & Tensor Parallelism (Model Parallelism)
- Научились считать Self-Attention для длинных контекстов



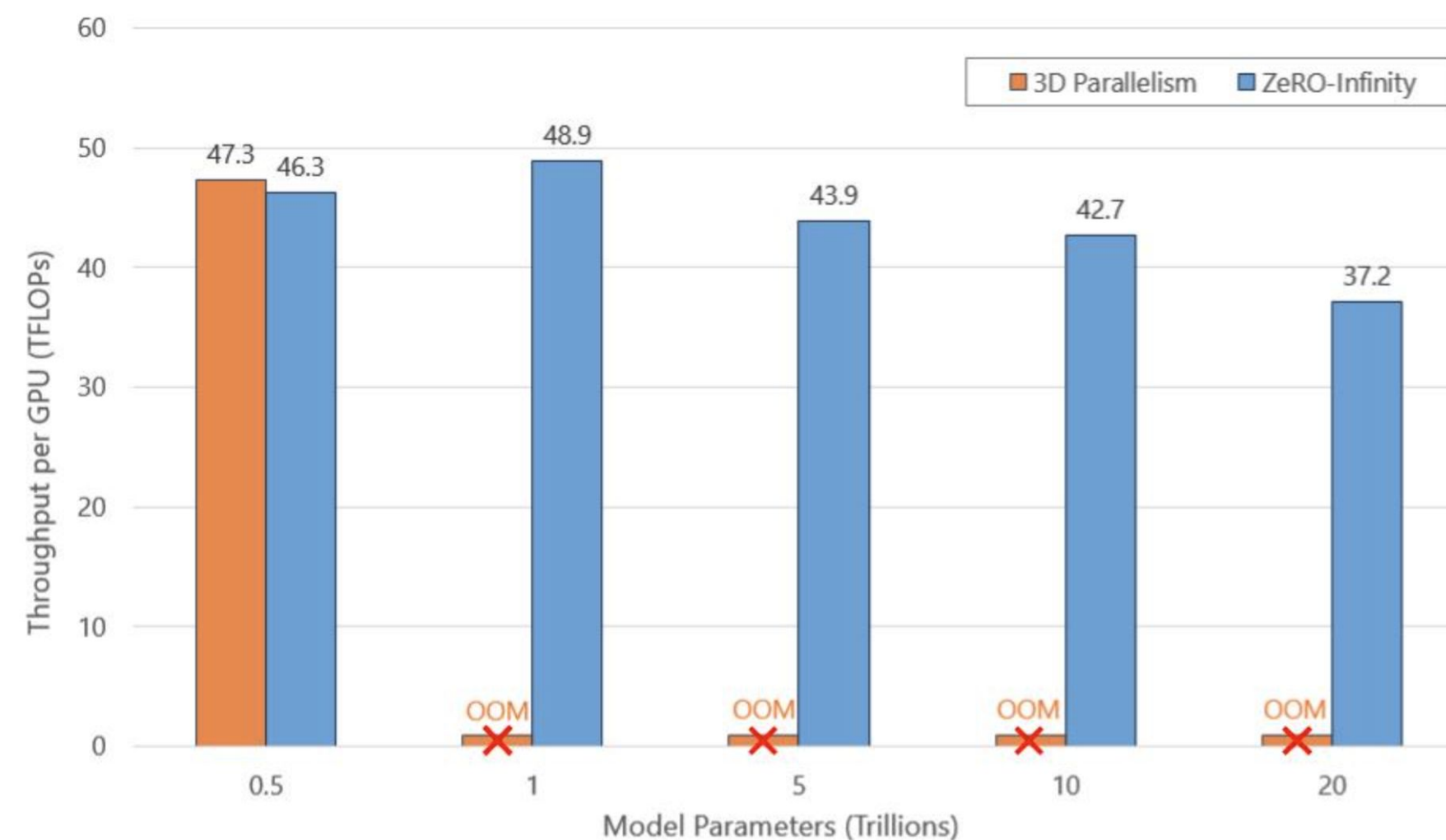
Резюме сегодняшней лекции

- Разобрали Pipeline & Tensor Parallelism (Model Parallelism)
- Научились считать Self-Attention для длинных контекстов
- Узнали как можно комбинировать различные виды параллелизма



Резюме сегодняшней лекции

- Разобрали Pipeline & Tensor Parallelism (Model Parallelism)
- Научились считать Self-Attention для длинных контекстов
- Узнали как можно комбинировать различные виды параллелизма
- Посмотрели как можно учить 10T+ модели



ССЫЛКИ

- GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism [Huang et al. 2018]
- Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [Shoeybi, et al, 2019]
- DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models [Jacobs, et al, 2023]
- Ring Attention with Blockwise Transformers for Near-Infinite Context [Liu, et al, 2023]
- Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM [Narayanan, et al, 2021]
- LongVILA: Scaling Long-Context Visual Language Models for Long Videos [Chen, et al, 2024]
- Efficient Training of Large Language Models on Distributed Infrastructures: A Survey [Duan, et al, 2024]
- ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning [Rajbhandari, et al, 2021]